

Politecnico di Torino

Politecnico di Torino

Corso di Laurea Magistrale in Ingegneria Informatica

A.A. 2025/2026

Sessione di laurea Dicembre 2026

GroundOne: un System One specialista per la verifica della groundedness nella generazione aumentata dal recupero in locale

Fine-tuning di modelli linguistici compatti come verificatori calibrati a singolo passaggio su Apple Silicon

Relatore: Prof. Paolo Garza

Candidato: Francesco Pio De Dominicis

Sommario

La generazione aumentata dal recupero (RAG) è il modo ormai consueto per dare ai modelli linguistici accesso a documenti privati, e l'esecuzione dell'intera pipeline sul dispositivo dell'utente è oggi praticabile su hardware di consumo, con vantaggi per la riservatezza, l'uso offline e i costi. Il recupero, tuttavia, non elimina l'allucinazione: le risposte generate contengono ancora frasi che il contesto recuperato non supporta o che lo contraddicono. Individuarle è un prerequisito per assistenti locali affidabili e, poiché il controllo deve essere eseguito su ogni risposta, esso deve trovare posto sulla stessa macchina dalle risorse limitate che ospita il generatore.

La verifica segue oggi due paradigmi. I verificatori basati su encoder, come AlignScore, MiniCheck e HHEM, sono piccoli e veloci, ma restituiscono un unico punteggio fisso per ogni coppia di documento e frase. I giudici generativi sono flessibili, ma scrivono i propri verdetti token per token, sicché il loro costo cresce con la quantità di testo prodotto; i giudici di piccole dimensioni violano spesso il formato di output, e la fiducia che esprimono è raramente utilizzabile come probabilità. Di recente è emersa una terza famiglia: i **modelli System One**, come il proprietario Jev e gli aperti SemIf e Rizzo-Flow, rispondono a domande chiuse in un unico passaggio in avanti, leggendo dai logit la probabilità di ciascuna opzione di risposta. Le evidenze a loro favore sono per lo più riportate dagli stessi autori, e le implementazioni aperte sono generaliste. Se un System One compatto, specializzato in un unico compito, possa eguagliare i giudici generativi e gli encoder dedicati, da dove provenga il suo vantaggio e se le sue probabilità siano utilizzabili, il tutto su un portatile, non era stato studiato in modo controllato.

Questa tesi progetta, addestra e valuta **GroundOne**, un System One specializzato in un'unica decisione: stabilire, per ogni frase di una risposta, se il documento recuperato la supporta, la contraddice o non la menziona. Ogni frase della risposta è seguita da un marcatore, e i verdetti per tutte le K frasi vengono letti dai logit di tre lettere di risposta in un unico passaggio in avanti, senza generare alcun testo. Modelli Qwen2.5 da 0.5B e 3B parametri sono adattati con LoRA sulle risposte di RAGTruth annotate da esseri umani, con una funzione di perdita pesata per classe e ristretta alle tre lettere; una temperatura e un bias stimati sui dati di sviluppo

calibrano le probabilità, e una soglia scelta sugli stessi dati le traduce in etichette coerenti e in segnalazioni rigorose o permissive. Ogni fase, dall'addestramento alla messa in servizio, viene eseguita con MLX su un portatile Apple M4 con 16 GB di memoria. La valutazione impiega tre insiemi di test costruiti e congelati prima dell'addestramento (la partizione di test di RAGTruth, LLM-AggreFact e VitaminC) e confronta GroundOne con indizi che non leggono mai il documento, con i modelli non addestrati, con giudici generativi della stessa dimensione con e senza probabilità, con un System One generalista e con encoder specializzati, riportando intervalli ottenuti tramite bootstrap a cluster.

Sull'insieme di test di RAGTruth GroundOne raggiunge un'accuratezza bilanciata a tre classi di 0.769 con 0.5B parametri e di 0.832 con 3B, contro 0.37 e 0.44 dei modelli non addestrati; la sua AUROC (0.905 e 0.940) supera quella degli encoder (circa 0.82), e il modello supera nettamente sia un System One generalista otto volte più grande sia un modello decisionale generalista della stessa dimensione (0.657), il quale risulta a sua volta molto più forte sull'insieme di test fuori dominio VitaminC. Su LLM-AggreFact il modello 3B è alla pari con MiniCheck-RoBERTa (74.8 contro 74.0, una differenza non significativa), si colloca al di sopra del modello 7B zero-shot pubblicato e al di sotto del System One chiuso Jev, misurato in modo indipendente (78.6). Per risposte di quattordici o più frasi GroundOne è circa due volte più veloce di un giudice sintetico e sei volte più veloce di un giudice verboso della stessa dimensione, con un output sempre valido; consuma inoltre meno energia (5.5 e 33 joule per risposta), sebbene con un fattore inferiore rispetto al tempo, poiché un prefill lungo assorbe più potenza della decodifica. Quando si richiede loro lo stesso output, i giudici generativi forniscono probabilità che distinguono a stento le decisioni corrette da quelle errate, mentre la calibrazione porta l'errore di calibrazione atteso di GroundOne sull'insieme di test a 0.019. A parità di pesi, restringere la decisione alle tre lettere è più accurato che prendere il token che il modello genererebbe, di 0.25 per il modello 0.5B e di 0.07 per il 3B. I limiti sono altrettanto chiari: il trasferimento a un dominio diverso è debole, le allucinazioni nascoste in frasi che per il resto ricalcano la fonte restano difficili da cogliere, e la ricetta di addestramento non è del tutto robusta rispetto al seed casuale.

Ringraziamenti

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Indice

Elenco delle figure VIII

1 Introduzione 1
- 1.1 Motivazione . . . 1
- 1.2 Modelli System One . . . 1
- 1.3 Domande di ricerca . . . 2
- 1.4 Contributi . . . 3
- 1.5 Struttura della tesi . . . 4

2 Contesto e lavori correlati 5
- 2.1 Inferenza dei transformer: prefill, decodifica e cache chiave-valore . 5
- 2.2 Inferenza sul dispositivo dell'utente . . . 6
- 2.3 Modelli System One . . . 7
- 2.4 Generazione aumentata dal recupero e allucinazione . . . 9
- 2.5 Verifica della groundedness . . . 9
- 2.6 Calibrazione e predizione selettiva . . . 10
- 2.7 Fine-tuning efficiente nei parametri . . . 11
- 2.8 Lacuna di ricerca . . . 11

3 GroundOne: metodo e implementazione 12
- 3.1 Compito e decisione . . . 12
- 3.2 Leggere tutti i verdetti in un solo passaggio in avanti . . . 13
- 3.3 Obiettivo di addestramento . . . 14
- 3.4 Calibrazione . . . 15
- 3.5 Software . . . 16

4 Dati 18
- 4.1 Compito ed etichette . . . 18
- 4.2 Dataset di origine . . . 19
- 4.3 Costruzione . . . 21
- 4.4 Il dataset risultante . . . 22

4.5 Affidabilità degli insiemi di test . . . 22
4.6 Analisi esplorativa . . . 24
4.7 Controllo della contaminazione (leakage) . . . 26

5 Impostazione sperimentale 28
5.1 Modelli . . . 28
5.2 Baseline . . . 29
5.3 Metriche . . . 31
5.4 Protocollo statistico . . . 32
5.5 Latenza, energia e hardware . . . 32

6 Risultati 34
6.1 Addestramento . . . 34
6.2 Variabilità tra seed . . . 35
6.3 Accuratezza sull'insieme di test RAG (T1) . . . 36
6.4 Confronto con le baseline . . . 38
6.5 Trasferimento a un dominio diverso (T3) . . . 40
6.6 Readout e generazione a confronto . . . 40
6.7 Calibrazione e predizione selettiva . . . 41
6.8 Probabilità a parità di output . . . 43
6.9 Latenza in funzione di K . . . 44
6.10 Energia per verifica . . . 46
6.11 Analisi degli errori . . . 46

7 Discussione, limiti e minacce alla validità 48
7.1 Quando conviene un System One specialista . . . 48
7.2 Limiti . . . 50
7.3 Minacce alla validità . . . 51

8 Conclusioni e sviluppi futuri 53
8.1 Sintesi dei risultati . . . 53
8.2 Sviluppi futuri . . . 54

A Output di ogni sistema su una stessa risposta 55
A.1 Verdetti frase per frase . . . 56
A.2 Output grezzi . . . 57

Bibliografia 59

Elenco delle figure

4.1 Distribuzione delle classi delle frasi etichettate in ciascuna partizione. RAGTruth (addestramento, sviluppo e T1) è supportato per circa l'88–91 %; T3 è campionato a 1,000 coppie per classe. 23
4.2 Distribuzione di K, il numero di frasi valutate per risposta, nell'addestramento e in T1. Le linee tratteggiate indicano le fasce usate per riportare i risultati in funzione di K. 23
4.3 Quota di frasi contraddette o non menzionate in base al modello che ha scritto la risposta, nell'addestramento e in T1. 25
4.4 Sovrapposizione lessicale tra una frase e il suo documento, per classe, nell'addestramento, in T1 e in T3 (media tra parentesi). 26
4.5 Quota di frasi contraddette o non menzionate in base alla posizione nella risposta. 27

6.1 Accuratezza bilanciata a tre classi sull'insieme di sviluppo durante l'addestramento del modello 0.5B, complessiva (a sinistra) e per compito (a destra). La linea punteggiata indica il livello del caso. Il 3B mostra lo stesso andamento sul suo sottoinsieme di sviluppo di 200 risposte. 35
6.2 Accuratezza bilanciata a tre classi sull'insieme di sviluppo del modello 0.5B durante l'addestramento, con la stessa ricetta e tre seed. Due esecuzioni convergono allo stesso livello; la terza è collassata su un predittore costante ed è stata interrotta. 36
6.3 T1: AUROC di P(supported) per ogni sistema (a sinistra) e accuratezza bilanciata a tre classi per i sistemi con output a tre classi (a destra). Rizzo-Flow è valutato su 300 risposte e il 3B non addestrato su 600; tutti gli altri sistemi, compreso il modello decisionale generalista d1-3B, sull'intero insieme di test. 38
6.4 Controllo B2 sull'intero T1: con gli stessi pesi e lo stesso passaggio in avanti, il verdetto viene letto dalle tre lettere di risposta (GroundOne) oppure preso come il token che la generazione greedy emetterebbe (i fallimenti di formato sono contati come errori). 41

6.5 Affidabilità di P(supported) del 0.5B prima (grezza) e dopo la calibrazione sull'insieme di sviluppo, per l'insieme di sviluppo, T1 e T3. I punti sono intervalli con almeno 20 frasi. 42
6.6 Curve rischio–copertura della decisione binaria del 0.5B con probabilità calibrate: tasso di errore man mano che le frasi meno sicure vengono rinviate. 43
6.7 Confronto a parità di output sulle 200 risposte di latenza: giudici generativi della stessa dimensione a cui si chiede l'output di GroundOne, con probabilità lette dai logit della lettera generata oppure scritte in JSON; tutti i sistemi sono calibrati sui dati di sviluppo con la stessa mappa. Solo frasi con un verdetto valido. 44
6.8 Tempo mediano per risposta in funzione di K sulle stesse 200 risposte di T1 (MLX in-process, una risposta alla volta, M4 con 16 GB), per GroundOne, il System One con una domanda per frase e i giudici generativi sintetico e verboso. Scala logaritmica. 45
6.9 Sul sottoinsieme di latenza: accuratezza bilanciata a tre classi (i fallimenti di formato sono contati come errori) e quota di frasi il cui verdetto non è stato possibile interpretare. 45
6.10 Energia per risposta verificata su 100 risposte di latenza (powermetrics, CPU, GPU e Neural Engine, al netto della potenza a riposo), con la potenza netta media durante l'esecuzione. 46

Capitolo 1

Introduzione

1.1 Motivazione

La generazione aumentata dal recupero (RAG) è diventata il modo standard per consentire a un modello linguistico di rispondere a domande su documenti che non ha mai visto durante l'addestramento: un sistema di recupero seleziona i passaggi pertinenti e il modello scrive una risposta tenendo tali passaggi nel proprio contesto [1]. Poiché modelli aperti compatti e runtime efficienti girano ormai su hardware di consumo [2, 3], l'intera pipeline può restare sul dispositivo dell'utente. Ciò è rilevante ogni volta che i documenti sono privati, che l'assistente deve funzionare senza rete o che il suo comportamento non deve cambiare a seguito di un aggiornamento del fornitore.

Il recupero non elimina l'allucinazione. Le risposte contengono ancora frasi che i passaggi recuperati non supportano, o che li contraddicono [4, 5], e non si può pretendere che un utente controlli ogni frase rispetto alla sua fonte. Un assistente locale affidabile richiede quindi un verificatore che controlli ogni risposta, frase per frase, sulla stessa macchina che l'ha generata. Un simile verificatore compete con il generatore per la stessa memoria, lo stesso tempo e la stessa batteria. La soluzione più diffusa, cioè chiedere a un modello linguistico di giudicare la risposta, è costosa proprio là dove un laptop è più debole: un giudice scrive i propri verdetti token per token, e su hardware locale ogni token generato costa molto più di un token letto. I giudici di piccole dimensioni, inoltre, violano il formato di output richiesto, e la confidenza che esprimono è raramente utilizzabile come probabilità.

1.2 Modelli System One

Un modo diverso di usare un modello linguistico per prendere decisioni ha attirato di recente l'attenzione con il nome di *modelli System One* [6]. Anziché generare una

risposta, un modello di questo tipo legge una sola volta la domanda e il contesto e restituisce una probabilità per ciascuna risposta ammissibile, ad esempio leggendo i logit delle lettere delle opzioni alla fine del prompt [7, 8]. Non viene generato alcun testo, l'output è sempre valido e molte domande sullo stesso contesto possono condividere un unico passaggio su di esso. L'idea si adatta bene alla verifica: una risposta di K frasi pone K domande chiuse sullo stesso documento, e una probabilità per ciascuna frase può essere sottoposta a soglia e calibrata.

Le evidenze a favore dei modelli System One, tuttavia, provengono per lo più dai loro stessi autori, e le implementazioni aperte sono sistemi generalisti che rispondono a qualunque domanda tipizzata. Restano aperte diverse questioni: se l'approccio sia vantaggioso per un compito specifico, come si collochi un modello specializzato in quel compito rispetto a uno generalista, ai giudici generativi e ai verificatori dedicati, e da dove derivi effettivamente l'eventuale vantaggio.

1.3 Domande di ricerca

Questa tesi progetta, addestra e valuta GroundOne, un modello System One compatto specializzato in un'unica decisione: stabilire, per ogni frase di una risposta RAG, se il documento recuperato la supporta, la contraddice o non la menziona. Il lavoro è guidato da cinque domande di ricerca.

RQ1, specializzazione. Un modello System One sottoposto a fine-tuning, con 0.5B o 3B parametri, può eguagliare modelli zero-shot più grandi e gli encoder specializzati nella verifica della groundedness (aderenza al documento) a livello di frase?

RQ2, lettura dei logit (readout) contro generazione. Quale contributo apporta il meccanismo System One a parità di dimensione del modello, rispetto ai giudici generativi che scrivono i propri verdetti e rispetto allo stesso modello che decide in base al token che genererebbe, e come cresce la differenza con il numero di decisioni per risposta?

RQ3, calibrazione. Le probabilità sono utilizzabili come confidenza, per fissare soglie e per rinviare le decisioni incerte, e come si confrontano con le probabilità che un giudice generativo è in grado di fornire?

RQ4, costo sul dispositivo. Il verificatore rientra nel budget di memoria, tempo ed energia di un laptop da 16 GB, e come si confronta il suo costo con quello di un giudice sulla stessa macchina?

RQ5, oltre un punteggio binario. Che cosa offre una decisione a tre classi per ogni frase che un verificatore binario non offre?

1.4 Contributi

La tesi apporta quattro contributi.

1. **GroundOne**, un modello System One specialista e aperto per la verifica della groundedness a livello di frase, eseguito interamente su un laptop. Legge tutti i verdetti di una risposta in un solo passaggio in avanti, grazie a un marcatore posto dopo ogni frase, ed è distribuito come pacchetto installabile con un'interfaccia a riga di comando e un'API HTTP locale, che include un'interfaccia compatibile con i client System One esistenti.

2. **Un dataset orientato al compito**, costruito a partire da annotazioni umane pubbliche, con tre insiemi di test validati senza nuove annotazioni e congelati prima dell'addestramento di qualsiasi modello, e con il leakage controllato a livello di documento.

3. **Una scomposizione controllata dei guadagni.** Il confronto separa il contributo della specializzazione, del readout ristretto e della dimensione del modello, rispetto a indizi che non verificano mai una frase, modelli non addestrati, giudici generativi con e senza probabilità, un modello System One generalista ed encoder specializzati, tutti sullo stesso hardware e con intervalli di cluster bootstrap.

4. **Uno studio di calibrazione, tempo ed energia**: come le probabilità possano essere calibrate e tradotte in etichette coerenti, quanto aiutino a rinviare le decisioni incerte, e come il tempo e l'energia per verifica crescano con la lunghezza della risposta.

In sintesi, i risultati mostrano che la specializzazione funziona nel dominio RAG: sull'insieme di test RAGTruth GroundOne raggiunge un'accuratezza bilanciata a tre classi di 0.769 con 0.5B parametri e di 0.832 con 3B, superiore a quella degli encoder specializzati, nettamente superiore a quella di un modello System One generalista otto volte più grande e superiore a quella di un modello decisionale generalista della stessa dimensione; su LLM-AggreFact, inoltre, il modello da 3B è alla pari con il miglior encoder di piccole dimensioni. Verifica le risposte lunghe circa due volte più velocemente di un giudice sintetico e sei volte più velocemente di un giudice verboso della stessa dimensione, con un output sempre valido, probabilità molto migliori e un minore consumo di energia. La restrizione del readout alle risposte ammissibili aumenta da sola l'accuratezza. I limiti sono altrettanto chiari: il trasferimento a un dominio diverso è debole, e lì il modello generalista risulta migliore; le allucinazioni nascoste in frasi che per il resto ricopiano la fonte restano difficili da individuare; la ricetta di addestramento, infine, non è del tutto robusta rispetto al seed casuale.

1.5 Struttura della tesi

Il Capitolo 2 passa in rassegna l'inferenza dei transformer, l'inferenza locale, i modelli System One, l'allucinazione nella generazione aumentata dal recupero, i metodi di verifica e la calibrazione. Il Capitolo 3 descrive GroundOne: la decisione, il readout in un solo passaggio, l'obiettivo di addestramento, la calibrazione e l'output servito. Il Capitolo 4 definisce le etichette e descrive come gli insiemi di addestramento e di test sono stati costruiti, verificati e congelati. Il Capitolo 5 presenta i modelli addestrati, le baseline, le metriche e il protocollo di misura. Il Capitolo 6 riporta i risultati, il Capitolo 7 ne discute il significato e i limiti, e il Capitolo 8 conclude. L'Appendice A mostra l'output di ciascun sistema su una stessa risposta.

Capitolo 2

Contesto e lavori correlati

Questo capitolo introduce le idee su cui si fonda GroundOne e i lavori con cui viene confrontato. Si parte da come un transformer impiega il proprio tempo durante l'inferenza, poiché il vantaggio di un modello System One ne discende direttamente, e da ciò che cambia quando l'inferenza viene eseguita sul dispositivo dell'utente. Si passano poi in rassegna i modelli System One, il problema dell'allucinazione nella generazione aumentata dal recupero (RAG) e i metodi che verificano le risposte rispetto alle loro fonti; il capitolo si chiude con la calibrazione, il fine-tuning efficiente nei parametri e la lacuna di ricerca che questa tesi affronta.

2.1 Inferenza dei transformer: prefill, decodifica e cache chiave-valore

Un modello linguistico di tipo decoder-only elabora il proprio input in due fasi. Nella fase di *prefill* l'intero prompt viene letto in parallelo: ogni strato calcola, per ciascun token di input, le chiavi e i valori a cui i token successivi presteranno attenzione, e li memorizza in una cache chiave-valore (KV cache). Nella fase di *decodifica* il modello produce il proprio output un token alla volta, e ogni nuovo token presta attenzione alla cache e la estende di una posizione [9]. Le due fasi si comportano in modo molto diverso sull'hardware. Il prefill esegue una grande quantità di calcolo su molti token contemporaneamente ed è limitato dalla capacità di calcolo aritmetico. La decodifica esegue poco calcolo per passo, ma deve leggere tutti i pesi, oltre alla cache in crescita, per ogni token generato, e dunque con batch piccoli è limitata dalla banda di memoria [9, 10]. Ne consegue che un token generato costa molto più tempo di un token del prompt, e su un singolo dispositivo la differenza raggiunge un ordine di grandezza. I sistemi di serving sfruttano questa distinzione in modi diversi, ad esempio gestendo la cache a pagine per accogliere

più richieste concorrenti [11] oppure eseguendo le due fasi su macchine separate, ciascuna adatta a una di esse [10].

La conseguenza pratica per la verifica è semplice. Un giudice che scrive il proprio verdetto paga un prefill per il documento e la risposta e poi un passo di decodifica per ogni token che scrive, si tratti di una sola lettera o di un paragrafo di ragionamento. Un modello che legge la propria decisione dai logit calcolati durante il prefill paga soltanto il primo costo. La cache chiave-valore offre una seconda leva: quando si pongono molte domande sullo stesso contesto, quest'ultimo può essere elaborato una sola volta e poi riutilizzato, cosicché ogni domanda aggiuntiva costa soltanto i suoi pochi token.

2.2 Inferenza sul dispositivo dell'utente

I modelli linguistici sono usati per lo più attraverso servizi remoti, che ricevono l'input dell'utente tramite la rete ed eseguono il modello in un datacenter. Eseguire il modello sul dispositivo dell'utente modifica diverse proprietà allo stesso tempo. I documenti non lasciano mai la macchina, il che è importante per dati privati o regolamentati; il sistema funziona senza rete e senza dipendere dalla disponibilità di un fornitore; il costo è rappresentato dall'hardware e dalla sua energia anziché da un prezzo per token; i pesi, e dunque il comportamento, non cambiano a meno che non sia l'utente a cambiarli. Un modello locale espone inoltre i propri logit, che le interfacce remote spesso nascondono o limitano e che sono indispensabili a un modello System One e alla sua calibrazione. Il prezzo da pagare è la capacità: un laptop può ospitare soltanto modelli piccoli, e deve ospitarli accanto a tutto ciò che vi è in esecuzione.

Tre proprietà dell'hardware determinano ciò che è fattibile. La capacità di memoria stabilisce quali modelli possono essere caricati: un modello da 7B in mezza precisione richiede più di 14 GB, oltre le possibilità della maggior parte dei dispositivi personali, ed è per questo che le installazioni locali si affidano a pesi quantizzati, a modelli più piccoli oppure, nei sistemi di ricerca, al caricamento progressivo dei pesi dalla memoria flash [12]. La banda di memoria limita la velocità della decodifica, poiché ogni token generato legge tutti i pesi; sul laptop utilizzato in questa tesi, un Apple M4 con 120 GB/s di banda e 16 GB di memoria unificata condivisa tra CPU e GPU [13], tale limite corrisponde a pochi millisecondi per token per un modello da 0.5B in precisione a 8 bit e a circa dieci volte tanto per un modello da 3B. La capacità di calcolo, infine, limita il prefill, che sulla GPU di un laptop è relativamente veloce.

Il software per l'inferenza locale è maturato rapidamente. llama.cpp [2] offre un'inferenza priva di dipendenze, con quantizzazione intera e back end per la maggior parte degli acceleratori di consumo, e molti strumenti di distribuzione

si basano su di esso. MLX [3], con il suo pacchetto per modelli linguistici [14], è un framework per array progettato per la memoria unificata di Apple silicon e supporta la quantizzazione e il fine-tuning LoRA direttamente sul dispositivo. I produttori di piattaforme distribuiscono ormai i propri modelli per l'inferenza locale (sul dispositivo): il foundation model di Apple per i suoi dispositivi è un modello denso di circa tre miliardi di parametri, quantizzato a meno di quattro bit per peso e specializzato per ciascuna funzionalità mediante adattatori a basso rango di rango 16 su tutti gli strati lineari [15]. Si tratta dello stesso schema, alla stessa scala, seguito da GroundOne.

L'energia è l'ultima dimensione dell'inferenza locale. Misure condotte su hardware da datacenter mostrano che l'energia di una richiesta cresce con il numero di token generati e dipende fortemente dalla dimensione del modello e dal compito [16, 17], e la distinzione tra un prefill limitato dal calcolo e una decodifica limitata dalla banda implica che anche la potenza assorbita dalle due fasi sia diversa [10]. Su un dispositivo alimentato a batteria l'energia per verifica è quindi una proprietà del metodo, non soltanto dell'hardware, e il Capitolo 6 la misura.

2.3 Modelli System One

Il termine *modello System One* è stato introdotto da TypeSafe con Jev, un modello offerto come servizio remoto che risponde a domande chiuse su uno stato dato con output tipizzati, come un booleano, una scelta tra opzioni o un punteggio, senza generare testo [6]. Il nome richiama la modalità di pensiero rapida e intuitiva descritta da Kahneman, contrapposta al ragionamento deliberato. Il fornitore dichiara latenze dell'ordine di decine o centinaia di millisecondi e forti risparmi di costo rispetto ai giudici generativi, ma queste cifre restano affermazioni e l'architettura non è resa nota.

Sono seguite implementazioni aperte, che differiscono nel modo in cui le probabilità vengono prodotte. La famiglia più diffusa mantiene la testa del modello linguistico e legge i logit delle lettere delle opzioni nella posizione della risposta, dopo un prompt che presenta la domanda come scelta multipla. SemIf applica questa lettura dei logit (readout) a un modello congelato da 4B e riporta che 21 criteri relativi a un documento ricevono risposta in circa un secondo, contro più di cinque secondi per la migliore baseline generativa in grado di produrre JSON valido [7]. Rizzo-Flow sottopone a fine-tuning con LoRA un modello da 4B su decisioni tipizzate, espone la stessa interfaccia di Jev e riporta miglioramenti di accuratezza e di calibrazione dopo il fine-tuning [8]; si tratta del modello System One generalista impiegato come baseline in questa tesi. Una seconda famiglia codifica separatamente lo stato e ciascuna risposta candidata e ne confronta gli embedding: i Contrastive Language Models sono molto veloci quando i candidati sono

numerosi e riutilizzabili, ma nel loro stesso confronto concordano con le decisioni di Jev molto meno di un modello che lascia interagire stato e candidato [18]. Una terza famiglia utilizza piccoli encoder con una testa decisionale dedicata. Laya, ad esempio, assegna un punteggio a ogni opzione in corrispondenza di un proprio token di maschera di un encoder ModernBERT di circa 400M parametri; le domande di una richiesta vengono valutate in un'unica chiamata a batch, ma ciascuna domanda viene codificata con una propria copia del contesto [19, 20].

Le evidenze a sostegno di questi sistemi sono disomogenee. Gran parte di esse proviene dagli autori stessi, spesso su una sola macchina e su un solo benchmark. Le valutazioni indipendenti sono recenti e meno favorevoli ai piccoli modelli aperti: una valutazione di Jev su 37 dataset trova ben calibrate le sue probabilità di scelta, mentre le sue probabilità binarie, pur ordinando bene gli esempi, risultano mal posizionate rispetto a una soglia fissa di 0.5, e riporta un'accuratezza bilanciata media di 78.6 su LLM-AggreFact [21]; una riproduzione indipendente di Laya rileva una calibrazione diversa da quella dichiarata e una forte dipendenza da una temperatura stimata sui dati di destinazione [22]; benchmark contrastivi con protocolli chiari collocano i piccoli modelli aperti molto al di sotto di Jev [23]; ed è stato mostrato che i modelli decisionali tipizzati seguono l'etichetta di un'opzione anziché la sua definizione quando le due sono in conflitto [20]. Nel 2026 l'hub di Hugging Face ha introdotto un tag per questi *modelli decisionali* [24], che a ottobre 2026 raggruppava già circa 1,500 modelli di circa 600 autori, di qualità molto diversa.

A poche settimane dal rilascio di Jev, organizzazioni affermate hanno pubblicato propri modelli decisionali aperti. Clef di Cloudflare aggiunge una testa decisionale dedicata a un decoder e risponde a tutte le domande in un solo passaggio sullo stato, l'architettura che per Jev era stata soltanto ipotizzata [25]; d1-3B di Liquid AI legge lettere di risposta ristrette da un modello da 3B il cui stato è condiviso da tutte le domande [26]; Perplexity sottopone a fine-tuning un modello da 27B con una testa sulle lettere [27]. Una classifica (leaderboard) della comunità, il Decision Index, valuta questi modelli su un insieme comune di benchmark pubblici che comprende RAGTruth a livello di risposte intere [28]. I suoi risultati tracciano una linea netta: i modelli decisionali generalisti con meno di un miliardo di parametri superano a malapena la baseline banale su RAGTruth, e soltanto modelli da decine di miliardi di parametri individuano in modo affidabile le risposte allucinate. Gli studi su Jev aggiungono che le sue probabilità sono calibrate quando aggregate su molti compiti, ma non su ciascun compito preso separatamente, che soglie e temperature vanno stimate sul carico di lavoro di destinazione [29, 30], e che scegliere una soglia mediante un limite inferiore di confidenza rende più sicuro l'impiego di un giudice che restituisce soltanto decisioni [30]. Anche i runtime si sono adeguati: llama.cpp supporta ora nativamente l'interfaccia System One [2]. Nessuno di questi modelli, tuttavia, è specializzato nella groundedness (aderenza al documento), e nessuno

riporta risultati di verifica a tre classi a livello di frase sui benchmark utilizzati in questa tesi.

Il readout sulle lettere è stato studiato anche al di fuori della letteratura sui modelli System One. Nella valutazione a scelta multipla, la probabilità del primo token di risposta e la risposta che un modello addestrato a seguire istruzioni scrive effettivamente sono spesso in disaccordo [31], e il readout è distorto a favore di particolari identificatori di opzione, un bias che può essere stimato e rimosso [32]. Entrambi i risultati riguardano modelli zero-shot e suggeriscono che un readout sulle lettere usato come decisione debba essere addestrato, calibrato o entrambe le cose. Nessun modello System One aperto è stato specializzato nella verifica della groundedness e confrontato, in condizioni controllate e sullo stesso hardware, con giudici generativi, verificatori specializzati e un modello decisionale generalista della stessa dimensione.

2.4 Generazione aumentata dal recupero e allucinazione

La generazione aumentata dal recupero consente a un modello linguistico di accedere a documenti su cui non è stato addestrato: un sistema di recupero seleziona i passaggi pertinenti alla domanda e il modello scrive la propria risposta tenendo questi passaggi nel contesto [1]. L'approccio è il modo standard per costruire assistenti su documenti privati e può essere eseguito interamente su una macchina locale. Non elimina, tuttavia, l'allucinazione. Le rassegne distinguono gli errori di *fattualità*, relativi alla conoscenza del mondo, dagli errori di *fedeltà*, relativi all'input o al contesto forniti [5]; una risposta RAG può essere infedele proprio ai passaggi che le sono stati forniti, contraddicendoli o aggiungendo informazioni che essi non contengono. RAGTruth documenta quanto ciò sia frequente: raccoglie circa 18,000 risposte scritte da sei modelli linguistici in tre compiti aumentati dal recupero e annota, a livello di intervalli di caratteri, le parti che sono in conflitto con il contesto o che non sono supportate da esso [4]. La sua distinzione tra contenuto in conflitto e contenuto privo di fondamento corrisponde alle classi *contraddetta* e *non menzionata* adottate in questa tesi.

2.5 Verifica della groundedness

Verificare una risposta rispetto alla sua fonte è strettamente legato all'inferenza in linguaggio naturale (NLI), il compito di stabilire se una premessa implica un'ipotesi, la contraddice o è neutrale rispetto a essa. Grandi dataset di inferenza come MultiNLI [33] e l'avversariale ANLI [34] forniscono dati di addestramento, e VitaminC [35] aggiunge coppie contrastive in cui una revisione minima di una frase

di Wikipedia ribalta il verdetto, costringendo il modello a guardare al fatto preciso anziché alla sovrapposizione di parole. Applicare questi modelli a testi generati lunghi richiede una scomposizione: SummaC suddivide in frasi sia il documento sia il riassunto e aggrega i punteggi di inferenza a livello di frase [36], mentre FActScore scompone le generazioni lunghe in fatti atomici e verifica ciascuno di essi [37]. La verifica a livello di frase, come in GroundOne, rappresenta una via di mezzo: le frasi sono unità naturali di una risposta, un utente può intervenire su una frase segnalata, e il numero di decisioni cresce con la lunghezza della risposta.

Nella pratica dominano due famiglie di verificatori. Gli encoder specializzati sono piccoli e veloci. AlignScore addestra un'unica funzione di allineamento su un'ampia miscela di compiti [38]; MiniCheck addestra piccoli verificatori su dati sintetici progettati per richiedere un ragionamento su più frasi ed eguaglia modelli molto più grandi sui benchmark di grounding [39]; HHEM è un modello di valutazione delle allucinazioni dello stesso tipo, rilasciato apertamente [40]. Questi modelli valutano in modo indipendente ciascuna coppia di documento e frase e restituiscono un'unica probabilità di supporto. La seconda famiglia impiega un LLM come giudice [41]. Framework di valutazione RAG come RAGAS [42] e ARES [43] si basano su giudici, Lynx sottopone a fine-tuning un giudice specificamente per l'individuazione delle allucinazioni [44], e Self-RAG addestra il generatore stesso a emettere token di critica [45]. I giudici sono flessibili, ma il loro costo cresce con ciò che scrivono, i modelli piccoli violano spesso il formato di output richiesto, e per garantire una risposta analizzabile è necessaria una decodifica vincolata da una grammatica [46]. LLM-AggreFact raccoglie undici dataset di grounding etichettati da esseri umani sotto un protocollo binario comune ed è il benchmark di riferimento per entrambe le famiglie [39, 47].

2.6 Calibrazione e predizione selettiva

Una probabilità è calibrata quando, tra le predizioni formulate con una data confidenza, la quota di predizioni corrette coincide con tale confidenza. Le reti neurali moderne sono spesso mal calibrate, e un'unica temperatura applicata ai logit, stimata su dati tenuti da parte, costituisce una correzione semplice ed efficace [48]; l'errore di calibrazione atteso (ECE), calcolato su intervalli di confidenza, è la misura abituale [49]. Per i modelli linguistici interrogati tramite prompt può essere necessario anche un bias per classe, che corregga una preferenza sistematica per alcune risposte [50]. I modelli linguistici sanno, in una certa misura, ciò che sanno: le loro probabilità sui token in risposta a domande chiuse sono spesso informative sulla correttezza [51]. Quando le probabilità non sono accessibili, la confidenza può essere richiesta a parole; per i modelli di grandi dimensioni sottoposti a fine-tuning con feedback umano questa confidenza verbalizzata può risultare meglio calibrata

delle probabilità stimate da campioni [52], ma valutazioni sistematiche trovano la confidenza verbalizzata grossolana ed eccessivamente sicura, con valori concentrati su pochi numeri tondi [53]. Una probabilità calibrata diventa utile attraverso la predizione selettiva: un sistema agisce sulle decisioni di cui è sicuro e rinvia le altre, e la curva rischio–copertura mostra come il tasso di errore diminuisca man mano che più decisioni vengono rinviate [54].

2.7 Fine-tuning efficiente nei parametri

Specializzare un modello preaddestrato non richiede di aggiornarne tutti i pesi. L'adattamento a basso rango (LoRA) congela il modello e addestra, per matrici di pesi selezionate, un aggiornamento a basso rango che può essere fuso nei pesi o mantenuto come piccolo adattatore separato [55]; applicarlo a tutti gli strati lineari conta più di un rango elevato, e il metodo si combina con pesi di base quantizzati per far entrare modelli grandi in poca memoria [56]. La famiglia Qwen2.5 mette a disposizione modelli aperti a partire da 0.5B parametri con un tokenizer condiviso, il che consente di confrontare dimensioni diverse del modello con la stessa ricetta [57]. Il fine-tuning su dataset piccoli è tuttavia notoriamente instabile: i risultati variano con il seed casuale, sia attraverso l'inizializzazione dei nuovi parametri sia attraverso l'ordine dei dati di addestramento [58]; alcune esecuzioni falliscono del tutto, un comportamento legato a dettagli dell'ottimizzazione come l'assenza della correzione del bias in Adam [59]; e il clipping della norma del gradiente è la difesa standard contro occasionali aggiornamenti esplosivi [60].

2.8 Lacuna di ricerca

Gli elementi necessari per un verificatore locale, veloce e calibrato esistono separatamente: modelli aperti compatti e runtime che li eseguono su un laptop, l'idea System One di leggere le decisioni anziché generarle, verificatori specializzati e calibrazione a posteriori. Ciò che manca è uno studio indipendente e controllato che li metta insieme per un unico compito. In particolare, nessun lavoro ha misurato se un modello System One compatto, specializzato nella verifica della groundedness, possa eguagliare giudici generativi ed encoder dedicati in quel compito, da dove provenga il suo vantaggio, tra specializzazione, readout e dimensione del modello, se le sue probabilità siano sufficientemente buone da fissare soglie e rinviare decisioni, e come il suo tempo e la sua energia si confrontino con quelli di un giudice sullo stesso hardware di consumo. Il resto di questa tesi affronta queste domande.

Capitolo 3

GroundOne: metodo e implementazione

GroundOne è un modello linguistico compatto impiegato come System One: legge una sola volta un documento e una risposta e restituisce un verdetto per ciascuna frase valutata della risposta, leggendo i logit di poche lettere di risposta senza generare testo. Questo capitolo descrive la decisione che il modello prende, il prompt che raccoglie tutte le decisioni di una risposta in un solo passaggio in avanti, l'obiettivo di addestramento, la calibrazione delle probabilità di output, la regola che trasforma le probabilità in etichette restituite e il software che realizza l'insieme.

3.1 Compito e decisione

Per ogni frase di una risposta GroundOne prende una decisione fra tre classi, definite nella Sezione 4.1: rispetto al documento, la frase è *supportata*, *contraddetta* o *non menzionata*. Le tre classi corrispondono alle lettere A, B e C, e il modello produce una probabilità per ciascuna di esse. Il punteggio binario di groundedness e le regole stretta e permissiva non costituiscono domande separate poste al modello: vengono ricavati da queste tre probabilità, cosicché un unico passaggio in avanti serve ogni utilizzo. Chiedere direttamente le regole non porterebbe alcun vantaggio. Ciascuna di esse è infatti una funzione deterministica della classe assegnata nella decisione a tre classi; domande separate moltiplicherebbero quindi le decisioni senza aggiungere informazione e potrebbero fornire al modello obiettivi incoerenti per la stessa frase.

3.2 Leggere tutti i verdetti in un solo passaggio in avanti

Il prompt contiene una breve istruzione, il documento e la risposta. Ogni frase della risposta è numerata, e ogni frase a cui è associata una decisione è seguita da un marcatore, la stringa “ `Verdict:`”. Le frasi prive di contenuto verificabile (Sezione 4.3) restano nel testo come contesto ma non ricevono alcun marcatore. Il modello elabora l'intera sequenza una sola volta; in corrispondenza dell'ultimo token di ciascun marcatore si leggono i logit delle lettere A, B e C, normalizzati con una softmax ristretta a queste tre lettere. Poiché un decoder presta attenzione soltanto alle posizioni precedenti, il verdetto sulla frase i dipende dal documento e dalle frasi $1, \ldots, i$, ed è proprio ciò che la verifica richiede: i riferimenti, come i pronomi, rimandano all'indietro. Nulla di quanto segue l'ultimo marcatore influisce sui verdetti, perciò la sequenza termina lì.

Due accorgimenti rendono la lettura dei logit (readout) efficiente ed esatta. In primo luogo, il prompt si costruisce tokenizzando separatamente le sue parti e concatenandole, così che la posizione di ogni marcatore sia nota con esattezza e coincida fra addestramento e inferenza. In secondo luogo, la proiezione sul vocabolario viene applicata soltanto nelle posizioni dei marcatori. Il corpo del transformer opera sull'intera sequenza, ma la grande matrice del vocabolario viene moltiplicata solo per i pochi stati nascosti che vengono letti.

Questa scelta determina il costo di una verifica. Se S è la lunghezza di documento e risposta, K il numero di decisioni e m la lunghezza in token di un marcatore, il System One richiede un unico prefill di $S + Km$ token. Un giudice generativo deve invece anche decodificare i propri verdetti: con n token per verdetto e un tempo di decodifica per token t_{dec}, il suo costo è all'incirca $\text{prefill}(S) + K\,n\,t_{\text{dec}}$. Poiché m vale tre token e, sull'hardware di riferimento, la decodifica è per token molto più lenta del prefill, il divario cresce al crescere di K. Sull'M4 impiegato in questa tesi il prefill costa circa 0.37 ms per token con il modello 0.5B e 2.2 ms con il 3B, mentre la decodifica costa 5.9 e 31.8 ms per token generato: un token decodificato costa quanto quattordici-sedici token elaborati in prefill. Le due fasi sollecitano inoltre l'hardware in modo diverso. Il prefill elabora molti token alla volta ed è limitato dal calcolo, mentre la decodifica produce un token alla volta ed è limitata dalla banda di memoria necessaria a leggere i pesi [9, 10]. Questa differenza ritorna nelle misure di energia del Capitolo 6.

Una seconda soluzione legge una sola volta il prefisso condiviso e aggiunge poi una breve domanda per ciascuna frase, valutando le domande in batch. Essa richiede una cache chiave-valore (KV cache) replicata K volte, mentre la soluzione a marcatori richiede un solo passaggio. Questa seconda soluzione è mantenuta come ripiego ed è il formato naturale per un modello non addestrato, che comprende una

domanda esplicita ma non ha motivo di interpretare un marcatore come tale; per questa ragione è utilizzata dalla baseline non addestrata (Sezione 5.2).

Un classificatore, non un generatore più veloce. Il readout restringe la decisione a tre lettere, e questa restrizione è rilevante. Dopo il marcatore, il modello addestrato non scriverebbe la lettera che viene letta: il token successivo più probabile è la stessa lettera preceduta da uno spazio, e le tre lettere del readout raccolgono insieme solo circa il 4–5 % della massa di probabilità in corrispondenza dei marcatori. L'addestramento con una funzione di perdita (loss) ristretta a queste lettere modella i loro logit relativi, non la loro probabilità assoluta, cosicché il readout si comporta come un classificatore a tre classi posto sopra il modello linguistico. È questo che definisce un System One: la risposta non viene mai decodificata, ma scelta fra le opzioni ammissibili. Ciò richiama anche l'osservazione secondo cui la probabilità del primo token di risposta e la risposta effettivamente scritta da un modello possono divergere [31], e quella secondo cui la scelta fra lettere porta con sé distorsioni di selezione [32], che un readout specializzato e calibrato deve assorbire. Un esperimento di controllo, che con gli stessi pesi e lo stesso passaggio prende il token che la generazione emetterebbe al posto del readout ristretto, quantifica la differenza (Sezione 6.6).

3.3 Obiettivo di addestramento

GroundOne si ottiene da Qwen2.5-0.5B-Instruct e Qwen2.5-3B-Instruct [57] mediante adattamento a basso rango (LoRA) [55]. Adattatori di rango 16, con fattore di scala 2 e dropout 0.05, sono applicati a tutti gli strati lineari di tutti i blocchi del transformer, poiché coprire ogni strato conta più di un rango elevato [56]. I pesi di base restano congelati in precisione a 8 bit. Un esempio di addestramento corrisponde a una risposta con tutte le sue frasi valutate, e la funzione di perdita è calcolata in corrispondenza di ogni marcatore della risposta.

La funzione di perdita è un'entropia incrociata sulle sole tre lettere: il prompt non è mai supervisionato e il segnale di addestramento è costituito interamente dai verdetti. Poiché circa l'88 % delle frasi di addestramento è supportato, alle classi sono assegnati pesi tali che ciascuna contribuisca in egual misura alla funzione di perdita; con la distribuzione di addestramento ne risultano pesi di circa 0.38 per le frasi supportate, 7.5 per quelle contraddette e 4.4 per quelle non menzionate. Senza tali pesi un modello può raggiungere una perdita bassa rispondendo quasi sempre "supportata". Ne deriva una conseguenza utile: un modello che avesse appreso soltanto le frequenze delle classi resterebbe a una perdita di $\ln 3 \approx 1.10$, e una perdita inferiore a questo valore indica quindi che il testo viene effettivamente utilizzato.

L'ottimizzazione impiega AdamW senza weight decay e, come nella configurazione predefinita di MLX, senza correzione del bias delle stime dei momenti; i gradienti non sono sottoposti a clipping. Il tasso di apprendimento (learning rate) di 10^{-4} viene raggiunto dopo un warm-up lineare sul primo 3 % dei passi e viene poi ridotto fino a zero con una schedulazione coseno, nell'arco di un'epoca sulle risposte di addestramento. Le risposte di lunghezza simile sono raggruppate in micro-batch entro un budget di token con padding, e i gradienti sono accumulati in modo che ogni passo dell'ottimizzatore veda circa 16 risposte, ossia circa 110 decisioni; in questo modo ogni passo contiene alcune frasi contraddette nonostante la loro rarità. Il gradient checkpointing mantiene la memoria entro i 16 GB del portatile. Il modello viene valutato sull'insieme di sviluppo a intervalli regolari, e si conserva il checkpoint con la più alta accuratezza bilanciata a tre classi, mediata sui tre compiti. La Sezione 5.1 riporta le impostazioni di ciascuna esecuzione.

3.4 Calibrazione

I pesi delle classi addestrano il modello come se le tre classi fossero ugualmente frequenti, sicché le sue probabilità grezze non sono calibrate sulla frequenza reale degli errori. Sull'insieme di sviluppo si adatta quindi una trasformazione a posteriori: i tre logit sono divisi per una temperatura T e traslati di un bias per classe, $\mathrm{softmax}(z/T + b)$, con il bias della classe supportata fissato a zero, e T e b minimizzano la log-verosimiglianza negativa delle etichette di sviluppo [48]. Il bias svolge il ruolo della correzione contestuale proposta per i modelli linguistici interrogati tramite prompt [50], qui stimata su dati etichettati. La sola temperatura modifica le probabilità ma non altera né la classe più probabile né l'ordinamento delle frasi. Il bias, invece, può cambiare la classe più probabile e, con tre classi, può anche riordinare leggermente $P(\text{supported})$ fra le frasi. Per questa ragione le metriche di classificazione e l'AUROC sono calcolate sull'output grezzo, mentre le probabilità calibrate si usano dove una probabilità deve esprimere una frequenza: per le metriche di calibrazione, per le soglie e per la predizione selettiva (Sezione 6.7).

La calibrazione è misurata principalmente con l'errore di calibrazione atteso (ECE) [49, 48]. Le predizioni sono suddivise, in base alla loro confidenza, in M intervalli di uguale ampiezza, con $M = 15$ in questa tesi. Per ogni intervallo B_m la confidenza media $\mathrm{conf}(B_m)$ è confrontata con la frequenza osservata $\mathrm{acc}(B_m)$, e gli scarti sono mediati con pesi proporzionali alla numerosità degli intervalli:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|. \tag{3.1}$$

Nella variante binaria la confidenza è $P(\text{supported})$ e la frequenza osservata è la quota di frasi supportate nell'intervallo; nella variante top-label a tre classi la

confidenza è la probabilità più alta e la frequenza osservata è la quota di predizioni corrette. L'ECE non confronta la distribuzione delle classi predette con quella dell'insieme di test, ma chiede se, fra le frasi a cui il modello assegna una data confidenza, tale confidenza corrisponda alla realtà. Essa dipende dalla scelta degli intervalli, e un predittore costante che restituisce sempre la frequenza di base ottiene un ECE prossimo a zero pur non discriminando nulla. Per questi motivi l'ECE è sempre riportato insieme al punteggio di Brier, alla log-verosimiglianza negativa e ai diagrammi di affidabilità, mentre la capacità delle probabilità di separare le decisioni corrette da quelle errate è misurata a parte, con l'AUROC e con l'area sotto la curva rischio–copertura.

Dalle probabilità alle etichette restituite. Chi utilizza il verificatore vede, per ogni frase, un'etichetta e una probabilità, e le due devono essere coerenti. La decisione grezza, ossia la classe più probabile dell'output non calibrato, è quella valutata in questa tesi, ma può essere in disaccordo con la probabilità calibrata mostrata accanto: una frase può ricevere l'etichetta "non menzionata" mentre la sua $P(\text{supported})$ calibrata supera di poco un mezzo. L'output restituito ricava perciò ogni elemento dalle probabilità calibrate tramite un'unica regola. Una frase riceve l'etichetta supportata se e solo se $P(\text{supported}) \geq \tau$, e altrimenti riceve la più probabile fra le due classi restanti; la regola stretta segnala ogni frase non supportata, mentre la regola permissiva segnala soltanto quelle contraddette. La soglia τ è stimata sull'insieme di sviluppo in modo da massimizzare l'accuratezza bilanciata a tre classi, e l'utente può modificarla al momento della richiesta per scegliere un altro punto operativo senza riaddestrare il modello.

3.5 Software

GroundOne è realizzato come pacchetto Python installabile basato su MLX [3, 14], il framework per array pensato per Apple silicon, e ogni fase di addestramento e di inferenza descritta finora viene eseguita al suo interno sullo stesso portatile. Il pacchetto comprende il costruttore del dataset con i relativi controlli, il ciclo di addestramento, la calibrazione, la valutazione sugli insiemi di test congelati, le baseline e una guardia di memoria che esegue i processi lunghi uno alla volta e arresta un processo prima che il sistema esaurisca la memoria. Il verificatore è accessibile tramite un'interfaccia a riga di comando e un server HTTP; il server offre un endpoint nativo e un endpoint compatibile con l'interfaccia adottata da altri modelli System One, così che i client esistenti possano usare GroundOne cambiando soltanto l'indirizzo. L'endpoint nativo riceve un documento e una risposta, segmenta la risposta con la stessa regola usata per costruire i dati, esegue l'unico passaggio in avanti della Sezione 3.2 e restituisce, per ogni frase, se è stata

valutata, le tre probabilità calibrate, l'etichetta restituita, i segnali delle regole stretta e permissiva, nonché la regola di decisione e la soglia che li hanno prodotti; riporta inoltre il numero di token in ingresso, il numero di token generati, che è sempre zero, e il tempo impiegato. L'endpoint compatibile risponde a generiche domande chiuse su un contesto condiviso, con il formato a una domanda per frase. Tutto il calcolo del modello avviene su un unico thread di lavoro di lunga durata, perché il contesto Metal della GPU non deve spostarsi fra thread, e nulla lascia la macchina: modelli, dati e richieste restano locali.

Capitolo 4

Dati

GroundOne apprende un'unica decisione, e i dati devono insegnare esattamente quella decisione e misurarla in modo affidabile. Il capitolo definisce dapprima la decisione e le sue etichette, per poi descrivere i tre dataset pubblici che le forniscono e il modo in cui vengono trasformati in insiemi di addestramento, di sviluppo e di test. Le ultime sezioni illustrano come l'affidabilità degli insiemi di test sia stata verificata senza nuove annotazioni, che cosa emerga da un'analisi esplorativa del risultato e come sia stata esclusa la contaminazione tra addestramento e test (leakage).

4.1 Compito ed etichette

L'ingresso è costituito da un documento recuperato e da una risposta scritta da un modello linguistico, suddivisa in frasi. Per ciascuna frase il modello stabilisce se, *rispetto al documento*, essa sia *supportata*, *contraddetta* o *non menzionata*. Una frase è supportata quando tutto ciò che afferma è detto o direttamente implicato dal documento. È contraddetta quando una sua parte è in conflitto con il documento, e non menzionata quando aggiunge informazioni che il documento né contiene né contraddice. Una frase che contiene sia una parte in conflitto sia una parte non verificabile riceve l'etichetta di contraddetta, poiché la contraddizione costituisce l'errore più grave. Conta soltanto il documento: un'affermazione vera nel mondo ma assente dal documento resta comunque non menzionata.

Il numero di decisioni relative a una risposta coincide con il numero delle sue frasi valutate, indicato con K. Questa grandezza è centrale per la tesi. Un System One legge il documento e la risposta una sola volta, sicché ogni decisione aggiuntiva ha un costo quasi nullo, mentre un giudice generativo deve produrre testo per ogni frase. Il confronto con i giudici generativi viene quindi condotto in funzione di K, e i dati devono contenere risposte lunghe oltre che brevi.

Il modello restituisce tre probabilità per frase, e ogni altra grandezza ne viene derivata anziché essere appresa separatamente. Il punteggio binario di groundedness (aderenza al documento), impiegato per il confronto con i benchmark binari pubblicati, è $P(\text{supported})$. Una regola *stretta*, secondo cui tutto ciò che non è affermato nel documento costituisce un errore, segnala una frase quando $P(\text{supported})$ scende al di sotto di una soglia τ. Una regola *permissiva*, secondo cui sono errori soltanto le contraddizioni, la segnala solo quando viene giudicata contraddetta. L'utente può così scegliere la nozione di errore, e la soglia, al momento dell'inferenza senza riaddestrare il modello; la Sezione 3.4 riporta la regola esatta adottata dal verificatore in servizio.

4.2 Dataset di origine

Il dataset è costruito deliberatamente a partire dal minor numero possibile di fonti: una per l'addestramento e tre per il test (Tabella 4.1). Un numero ridotto di fonti comporta un'unica definizione delle etichette, un unico formato testuale e meno punti in cui la costruzione possa introdurre errori.

Tabella 4.1: Dataset di origine e loro ruolo in GroundOne.

Dataset	Licenza	Ruolo	Etichette
RAGTruth [4]	MIT	addestramento, sviluppo, test T1	span di caratteri di quattro tipi di allucinazione su risposte di LLM
LLM-AggreFact [39, 47]	CC BY-ND 4.0	solo test T2	binarie (supportata / non supportata) per affermazione
VitaminC [35]	CC BY-SA 3.0	solo test T3	supporta / confuta / informazioni insufficienti

RAGTruth. RAGTruth [4] contiene 17,790 risposte scritte da sei modelli linguistici (GPT-3.5, GPT-4, Llama-2 7B, 13B e 70B, e Mistral-7B) in tre compiti di generazione aumentata dal recupero (RAG). Nella risposta a domande (QA) il contesto è un insieme di passaggi recuperati per una domanda di MS MARCO [61]. Nel riassunto si tratta di un articolo di cronaca tratto da CNN/DailyMail [62, 63] o da notizie recenti, mentre nel data-to-text (generazione da dati strutturati) è una scheda di un esercizio commerciale di Yelp in formato JSON [64]. Due annotatori formati hanno marcato gli span (porzioni di testo) di caratteri di ciascuna risposta non supportati dal contesto, con una terza revisione in caso di disaccordo, e hanno

assegnato a ogni span uno di quattro tipi: *conflitto* evidente o sottile, e *informazione infondata* evidente o sottile. L'articolo riporta un accordo del 91.8 % a livello di risposte e del 78.8 % a livello di span. Si tratta dell'unico dataset pubblico che unisca risposte naturali composte da più frasi, prodotte da diversi modelli, a una distinzione tra contenuto contraddetto e contenuto non supportato, ed è per questo l'unica fonte di addestramento. La sua partizione ufficiale separa 450 fonti di test dalle restanti fonti di addestramento.

LLM-AggreFact. LLM-AggreFact [39, 47] aggrega undici dataset di verifica dei fatti, etichettati da esseri umani, composti da coppie (documento, affermazione). Essi coprono il riassunto (AggreFact-CNN e XSum [65], TofuEval [66]), la risposta a domande aumentata dal recupero e in forma estesa (RAGTruth [4], ClaimVerify [67], ExpertQA [68], LFQA [69]) e la verifica dei fatti (WiCE [70], REVEAL [71], FactCheck-GPT [72]). La sua classifica pubblica rende i risultati confrontabili con i sistemi pubblicati. La licenza stabilisce che i dati non devono essere usati per addestrare modelli [73]; in questa tesi il dataset è quindi impiegato soltanto per la valutazione, e nessun addestramento, calibrazione o selezione avviene su alcuna delle sue partizioni.

VitaminC. VitaminC [35, 74] è un dataset di verifica dei fatti costruito a partire da revisioni di Wikipedia che modificano un fatto. Per ogni revisione, annotatori formati hanno scritto affermazioni che una versione della frase supporta e l'altra confuta; quando non era possibile formulare una contraddizione, la coppia è stata etichettata come priva di informazioni sufficienti. Poiché le due versioni della prova sono quasi identiche, un modello deve concentrarsi sul fatto preciso anziché sulla sovrapposizione lessicale. Su un campione ri-annotato di 2,000 coppie gli autori riportano un κ di Fleiss pari a 0.71. VitaminC svolge un ruolo specifico in questa tesi. RAGTruth non pubblica alcun accordo sul *tipo* di allucinazione, e da solo non può quindi dimostrare che il confine tra contraddetta e non menzionata venga appreso. VitaminC fornisce tale confine con etichette di affidabilità nota, in un dominio diverso.

Diversi altri candidati sono stati esaminati e non adottati. I corpora di inferenza in linguaggio naturale come ANLI [34] presentano premesse brevi, lontane dai contesti della generazione aumentata dal recupero, e VitaminC copre la stessa distinzione a tre classi con testo fattuale. I dati di addestramento sintetici o soltanto binari, come quelli rilasciati con MiniCheck [39], non contengono le etichette a tre classi. I dataset le cui etichette sono state generate da un modello linguistico, o che riutilizzano risposte di test di RAGTruth, sono stati esclusi per mantenere indipendenti gli insiemi di test e umane le etichette.

4.3 Costruzione

Segmentazione in frasi. Le risposte vengono dapprima suddivise in corrispondenza degli a capo, poiché contengono spesso elenchi e intestazioni, e ciascuna riga viene poi segmentata con PySBD [75], un segmentatore basato su regole che gestisce abbreviazioni e numeri decimali. I marcatori di elenco iniziali come "3." o "*" vengono rimossi, poiché riguardano l'impaginazione e non il contenuto. Lo stesso segmentatore è impiegato per l'addestramento, la valutazione e l'inferenza, così che il modello veda sempre frasi tagliate nello stesso modo.

Frasi prive di contenuto verificabile. Alcune frasi non formulano alcuna affermazione sul documento: saluti, intestazioni che introducono un elenco, righe contenenti soltanto una citazione come "(Passage 2)" e formule di rifiuto. Esse restano nella risposta numerata come contesto, ma non costituiscono decisioni e non ricevono etichetta. Una breve regola le individua e si applica all'8.2 % di tutte le frasi di RAGTruth, per lo più intestazioni di elenchi. La regola riguarda soltanto la formattazione, come i marcatori di elenco, le righe di sola citazione e le formule conversazionali, e non modifica mai un'etichetta; la sua copertura è stata verificata rispetto alla segmentazione indipendente di LLM-AggreFact descritta nella Sezione 4.5.

Etichette. Una frase valutata riceve l'etichetta di contraddetta se si sovrappone a uno span di conflitto, di non menzionata se si sovrappone soltanto a span di informazione infondata, e di supportata negli altri casi. Solo 27 dei 14,226 span annotati (0.19 %) non toccano alcuna frase valutata, e la conversione non comporta quindi quasi alcuna perdita di annotazioni. Per LLM-AggreFact l'etichetta binaria viene mantenuta così come rilasciata. Le etichette di VitaminC sono mappate direttamente: *supports* su supportata, *refutes* su contraddetta e *not enough information* su non menzionata.

Esclusioni e partizioni. Vengono rimosse le risposte che RAGTruth segnala come rifiuti scorretti o come troncate, così come le risposte prive di frasi valutate e quelle in cui documento e risposta superano insieme i 2,048 token, limite di lunghezza imposto dalla memoria del portatile da 16 GB. La partizione di test ufficiale di RAGTruth diventa l'insieme di test T1. Le fonti di addestramento vengono rimescolate con un seme fisso: il 10 % di esse forma l'insieme di sviluppo e il resto l'insieme di addestramento, in modo che risposte relative alla stessa fonte non compaiano mai in entrambi. Vengono rimosse sei risposte di addestramento relative a un articolo di cronaca presente anche nell'insieme di test di LLM-AggreFact. Dalla parte reale della partizione di test di VitaminC, che è sbilanciata, vengono campionate con un seme fisso 1,000 coppie per classe, che formano T3.

4.4 Il dataset risultante

La Tabella 4.2 riassume i cinque file. L'insieme di addestramento contiene 13,203 risposte e 90,795 frasi etichettate. I record sono memorizzati per risposta, con un'etichetta per frase, così che gli stessi dati possano alimentare modi diversi di leggere le K decisioni di una risposta.

Tabella 4.2: Il dataset GroundOne. Una decisione è una frase etichettata; T2 è binario.

Partizione	Fonte	Record	Decisioni	Supportate	Contraddette	Non menzionate
Addestramento	RAGTruth	13,203	90,795	79,898	4,039	6,858
Sviluppo	RAGTruth	1,468	9,996	8,827	410	759
T1	RAGTruth (test)	2,647	17,284	15,745	584	955
T2	LLM-AggreFact (test)	29,320	29,320	22,832	6,488 non supportate	
T3	VitaminC (test)	3,000	3,000	1,000	1,000	1,000

RAGTruth è fortemente sbilanciato: circa l'88 % delle frasi etichettate è supportato, e le contraddizioni sono soltanto dal 3 al 4 % (Figura 4.1). Si tratta della frequenza naturale degli errori nelle risposte dei modelli, e viene mantenuta: l'insieme di test riflette ciò che un verificatore incontra nella pratica, mentre lo sbilanciamento viene gestito durante l'addestramento. T3 compensa offrendo un test bilanciato a tre classi. Le risposte contano in media 6.9 frasi valutate nell'addestramento e 6.5 in T1, con un massimo di 28 (Figura 4.2). Per l'analisi in funzione della lunghezza, T1 è suddiviso in quattro fasce di K: 756 risposte con 1–4 frasi, 1,288 con 5–8, 497 con 9–13 e 106 con 14 o più. Le celle con meno di 30 frasi di una classe minoritaria, come le frasi contraddette nella risposta a domande, vengono riportate ma contrassegnate come indicative.

4.5 Affidabilità degli insiemi di test

Gli insiemi di test sono stati verificati senza nuove annotazioni, ricorrendo a prove indipendenti dalla costruzione sopra descritta (Tabella 4.3). Tutte le etichette di test provengono dagli annotatori umani dei dataset pubblicati. Le due scelte che potrebbero introdurre errori, ossia la segmentazione e la conversione dagli span alle etichette di frase, sono state confrontate con LLM-AggreFact, i cui autori hanno convertito in modo indipendente le stesse risposte di test di RAGTruth in affermazioni a livello di frase. Il segmentatore recupera esattamente il 96.3 % delle loro affermazioni, e soltanto l'8.5 % delle frasi valutate è assente dal loro insieme, per lo più frasi eliminate dal loro filtro non documentato delle affermazioni "non degne di verifica". Sulle 15,492 frasi presenti in entrambi, le etichette binarie concordano nel 98.75 % dei casi, e tutti i 193 disaccordi vanno nella stessa direzione: la regola basata

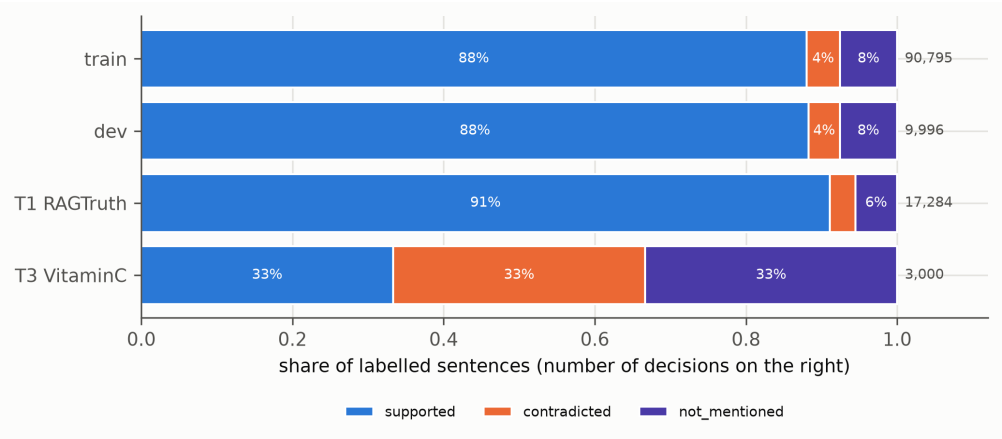


Figura 4.1: Distribuzione delle classi delle frasi etichettate in ciascuna partizione. RAGTruth (addestramento, sviluppo e T1) è supportato per circa l'88–91 %; T3 è campionato a 1,000 coppie per classe.

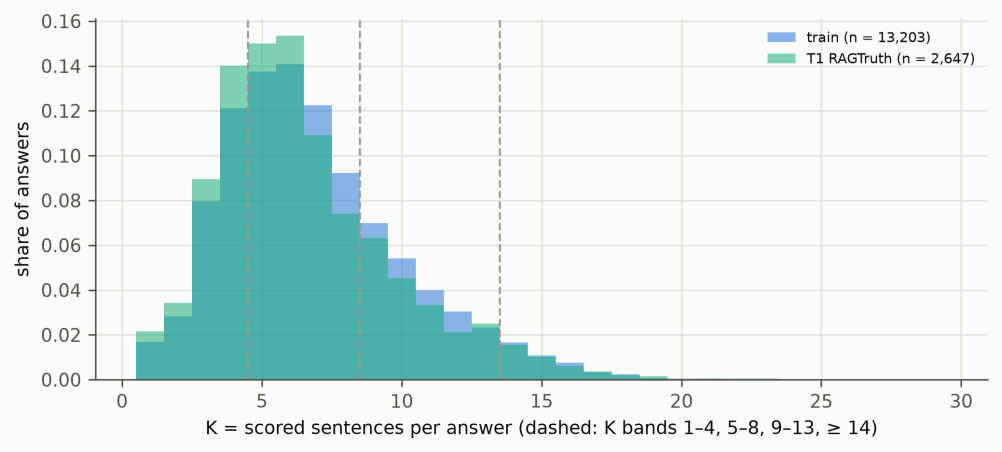


Figura 4.2: Distribuzione di K, il numero di frasi valutate per risposta, nell'addestramento e in T1. Le linee tratteggiate indicano le fasce usate per riportare i risultati in funzione di K.

sugli span marca come non supportata una frase che LLM-AggreFact considera supportata. Infine, un modello di riferimento basato su encoder, valutato su T2 senza modifiche, riproduce il proprio punteggio pubblicato (74.0 contro 73.5), il che convalida il codice di valutazione.

Tabella 4.3: Controlli di affidabilità eseguiti prima che gli insiemi di test venissero congelati.

Controllo	Criterio	Risultato
Provenienza delle etichette	ogni etichetta di test proviene da annotatori umani	superato
Segmentazione	$\geq$ 90 % delle affermazioni di LLM-AggreFact recuperate, $\leq$ 10 % delle frasi assenti da esso	96.3 % / 8.5 %
Derivazione delle etichette	accordo $\geq$ 97 % con le etichette binarie di LLM-AggreFact	98.75 %
Modello di riferimento	MiniCheck-RoBERTa su T2 entro 1.5 punti dal punteggio pubblicato	74.0 contro 73.5
Contaminazione	nessuna fonte, documento o coppia (documento, frase) condivisa tra addestramento e test	superato

Questi controlli convalidano il versante binario di T1. La distinzione tra contraddetta e non menzionata si fonda invece sugli annotatori di RAGTruth, il cui accordo sul tipo di allucinazione non è pubblicato. Inoltre, una recente ri-annotazione della partizione di test di RAGTruth ha individuato 1.68 volte più risposte allucinate rispetto alle etichette originali [76], il che suggerisce che T1 sottostimi le due classi minoritarie. Per questi motivi i risultati a tre classi su T1 vengono sempre letti insieme a quelli su T3. Dopo questi controlli, e prima che qualsiasi modello venisse addestrato, i tre file di test sono stati congelati mediante hash crittografici, e il costruttore del dataset si rifiuta di sovrascriverli con contenuti diversi; ricostruire il dataset a partire dalle fonti grezze riproduce esattamente i file congelati.

4.6 Analisi esplorativa

Un'analisi esplorativa delle fonti e del dataset costruito ha messo in luce quattro proprietà che determinano il modo in cui i risultati vanno letti.

Gli errori dipendono dal compito e dal modello. Le risposte di data-to-text contengono allucinazioni con una frequenza da due a quattro volte superiore a quella delle risposte negli altri compiti, e le risposte scritte da Llama-2 e Mistral

concentrano la maggior parte delle allucinazioni, mentre quelle di GPT-3.5 e GPT-4 sono quasi sempre fedeli (Figura 4.3). I risultati vengono quindi riportati per compito, poiché una singola media rifletterebbe soprattutto il data-to-text.

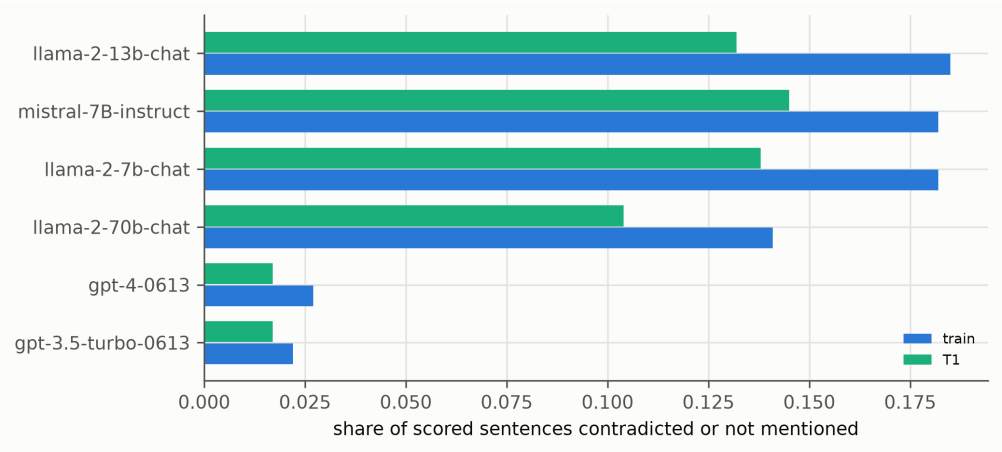


Figura 4.3: Quota di frasi contraddette o non menzionate in base al modello che ha scritto la risposta, nell'addestramento e in T1.

Le risposte di test sono più pulite di quelle di addestramento. In ogni compito, e per ogni generatore, T1 presenta una quota di frasi contraddette e non menzionate inferiore a quella dell'insieme di addestramento; nella risposta a domande, per esempio, è contraddetto lo 0.6 % delle frasi di T1 contro l'1.6 % nell'addestramento. Le metriche che dipendono dalla frequenza di base, come la precisione o l'F1 sulle classi minoritarie, non sono quindi confrontabili tra sviluppo e test. Le metriche principali sono l'accuratezza bilanciata, il richiamo macro e l'AUROC, che non ne dipendono.

Le contraddizioni sono per lo più evidenti. Circa l'85 % delle frasi contraddette di addestramento proviene da span annotati come conflitti *evidenti*, come numeri o nomi errati, e soltanto 147 da conflitti *sottili*. La classe contraddetta è dunque composta per lo più da contraddizioni palesi, e le contraddizioni sottili non possono essere valutate separatamente.

Gli indizi superficiali esistono e definiscono i modelli di riferimento. Due semplici indizi predicono le etichette senza verificare la frase rispetto al documento. Il primo è la sovrapposizione lessicale. La quota di parole di contenuto di una frase che compaiono anche nel documento è più alta per le frasi supportate che per quelle contraddette e non menzionate, in tutti e tre gli insiemi di test (Figura 4.4).

Da sola, essa separa le frasi supportate da quelle non supportate con un AUROC di 0.74 su T1 e di 0.70 su T3. Il secondo indizio è la formulazione della frase. Un classificatore basato sulla sola frase raggiunge un'accuratezza bilanciata a tre classi di 0.66 sul data-to-text, poiché alcuni dettagli, come il numero di recensioni o la disponibilità di Wi-Fi o di parcheggio con valet, vengono spesso inventati nelle descrizioni di Yelp. Nella risposta a domande e nel riassunto lo stesso classificatore resta al livello del caso (0.36 e 0.35), e così anche su T3 (0.33), dove il disegno contrastivo elimina l'indizio. Un verificatore è utile solo nella misura in cui supera questi indizi, e per questo entrambi vengono riportati come modelli di riferimento, per compito.

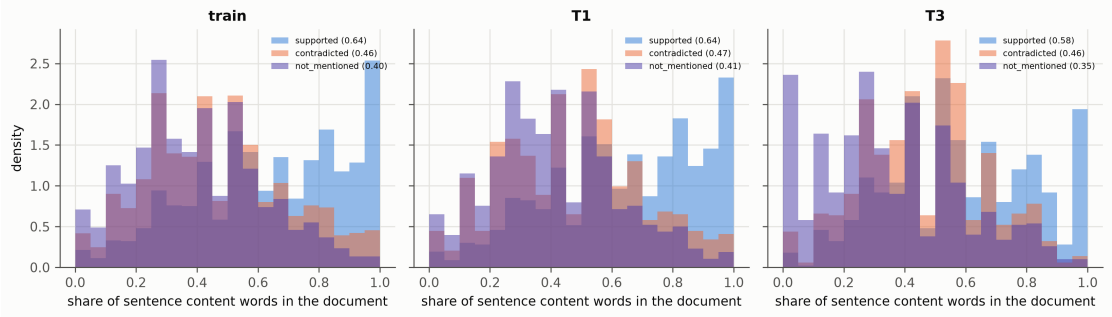


Figura 4.4: Sovrapposizione lessicale tra una frase e il suo documento, per classe, nell'addestramento, in T1 e in T3 (media tra parentesi).

Infine, le frasi non supportate non si concentrano alla fine della risposta. La prima frase è raramente non supportata, la quota raggiunge il massimo nelle posizioni dalla terza alla quinta e resta tra l'8 e l'11 % nelle posizioni successive (Figura 4.5). Anche le risposte lunghe contengono errori verso la fine, e l'accuratezza per valori elevati di K è quindi rilevante.

4.7 Controllo della contaminazione (leakage)

Le partizioni sono definite per fonte, mai per frase, e un controllo automatico viene eseguito a ogni costruzione. Il controllo fallisce se una fonte, un documento o una coppia (documento, frase) dell'insieme di addestramento o di sviluppo compare in uno qualsiasi degli insiemi di test, oppure se gli insiemi di sviluppo e di addestramento condividono una fonte. Il controllo risulta superato. Esso segnala inoltre le stringhe di frase che ricorrono sia nell'addestramento sia nel test con documenti diversi: si tratta di 788 frasi modello, come "The business has a rating of 4.0 stars.", attese nel data-to-text e che non rivelano alcuna etichetta di test. Non è possibile verificare se i dataset pubblici qui impiegati facessero parte dei dati

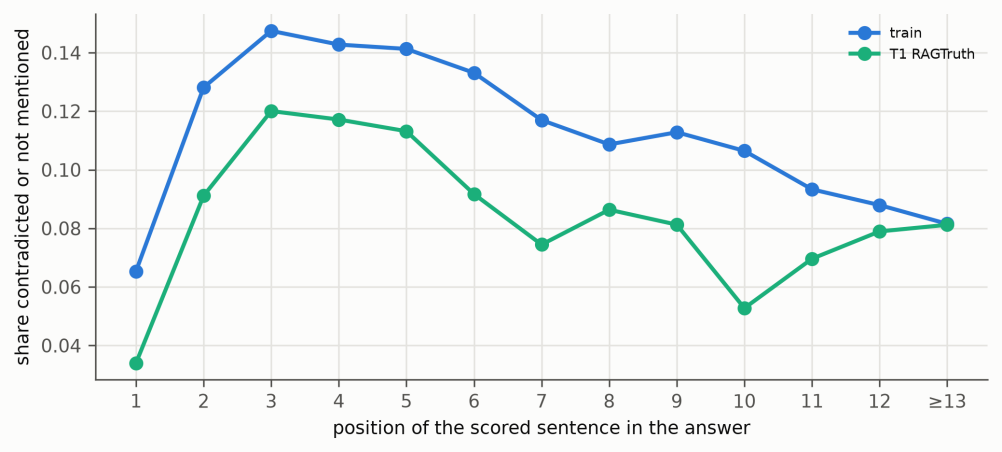


Figura 4.5: Quota di frasi contraddette o non menzionate in base alla posizione nella risposta.

di pre-addestramento di Qwen2.5, e questo resta un limite condiviso dalla maggior parte dei lavori basati su benchmark pubblici.

Capitolo 5

Impostazione sperimentale

Questo capitolo descrive i modelli addestrati, i sistemi con cui vengono confrontati, le metriche, il protocollo statistico e il modo in cui si misurano tempo ed energia. Tutti gli esperimenti sono eseguiti su un'unica macchina, un MacBook Pro con chip Apple M4 (CPU a 10 core, GPU a 10 core, 120 GB/s di banda di memoria) e 16 GB di memoria unificata [13]: in questo modo ogni risultato riflette ciò che un verificatore è in grado di fare sul dispositivo su cui gira un assistente locale. Anche ogni baseline, compresi gli encoder e il System One generalista, è eseguita su questo portatile; nessun risultato dipende da un servizio remoto.

5.1 Modelli

Entrambi i modelli GroundOne partono dai checkpoint di Qwen2.5 addestrati a seguire istruzioni [57], convertiti in MLX con pesi a 8 bit, ed entrambi sono addestrati con la procedura della Sezione 3.3. Differiscono soltanto per dimensione, per numero di risposte di addestramento e per il modo in cui si formano i micro-batch, aspetto dettato dalla memoria disponibile.

Il modello 0.5B è addestrato su tutte le 13,203 risposte di addestramento. Un'epoca di 802 passi dell'ottimizzatore ha richiesto 5.5 ore di tempo reale (wall-clock), con una valutazione sull'intero insieme di sviluppo ogni 150 passi, e l'addestramento ha occupato al massimo 2.6 GB di memoria. Il modello 3B è circa sei volte più lento per token, perciò viene addestrato per una notte su un sottoinsieme casuale di 6,500 risposte (circa 45,000 decisioni): 403 passi in 12.25 ore, con un picco di 5.9 GB. La sua valutazione sullo sviluppo, ogni 80 passi, usa un sottoinsieme fisso di 200 risposte, poiché un passaggio completo richiederebbe circa un'ora. In entrambi i casi si conserva il checkpoint con il miglior punteggio sullo sviluppo, che viene poi calibrato come descritto nella Sezione 3.4; per il 3B si tratta del passo 320.

Per misurare quanto i risultati dipendano dal seed casuale, che determina l'inizializzazione degli adapter e l'ordine dei batch, la procedura del 0.5B viene ripetuta con un secondo seed. È noto, infatti, che gli esiti del fine-tuning variano con entrambi i fattori [58]. Un'ulteriore esecuzione con un terzo seed è andata precocemente in collasso verso una predizione costante ed è stata interrotta a circa il 60 % dell'epoca; essendo incompleta, è esclusa dalle tabelle e viene discussa nella Sezione 6.2. La seconda esecuzione, portata a termine, è valutata su tutti e tre gli insiemi di test esattamente come la prima.

5.2 Baseline

Le baseline servono a isolare le fonti di un eventuale vantaggio, e ciascuna risponde a una domanda specifica.

Indizi che non leggono mai il documento nel suo insieme. Due predittori semplici stabiliscono la soglia minima che qualunque verificatore deve superare. Il primo assegna a una frase un punteggio pari alla quota delle sue parole di contenuto che compaiono anche nel documento. Il secondo è un classificatore bag-of-words addestrato soltanto sulle frasi di addestramento, senza il documento. Entrambi sono descritti nella Sezione 4.6.

Readout senza addestramento (B1). Agli stessi modelli Qwen2.5, privi di fine-tuning, viene posta la domanda a tre classi per ciascuna frase e se ne leggono i logit delle lettere di risposta (readout, ossia lettura dei logit). Poiché un modello non addestrato non ha motivo di interpretare il marcatore `Verdict:` come una domanda, questa baseline adotta il formato con una domanda esplicita per frase (Sezione 3.2). Essa misura ciò che il meccanismo System One offre prima di qualsiasi specializzazione.

Giudici generativi (B0a, B0b). I modelli non addestrati sono impiegati anche nel modo più comune nella pratica: ricevono il documento, la risposta numerata e l'elenco delle frasi da controllare, e scrivono un verdetto per ciascuna frase con decodifica greedy. Il giudice *sintetico* scrive una riga "`<number>: <letter>`" per frase, mentre il giudice *verboso* scrive una breve frase di ragionamento seguita dal verdetto. Un verdetto che non può essere interpretato costituisce un errore di formato; viene conteggiato come errore nell'accuratezza e riportato separatamente.

Giudici a parità di output (B0d). I giudici sintetico e verboso restituiscono meno di GroundOne: una lettera per frase e nessuna probabilità, e quindi non offrono calibrazione, soglia regolabile né scelta della politica. A due ulteriori

giudici si chiede pertanto esattamente il contenuto prodotto da GroundOne, cioè un'etichetta e tre probabilità per frase. Il primo li scrive come oggetto JSON, con le probabilità espresse a parole, che è ciò che uno sviluppatore otterrebbe con il solo prompt. Il secondo è il giudice sintetico le cui tre probabilità vengono lette dai logit in corrispondenza della lettera generata; ottiene lo stesso contenuto al costo del giudice sintetico ed è l'avversario generativo più forte. Entrambi i giudici sono calibrati con la stessa temperatura e lo stesso bias di GroundOne, stimati su 200 risposte di sviluppo, affinché le loro probabilità ricevano lo stesso trattamento. Il prompt JSON è stato scelto su dodici risposte di sviluppo unicamente in base alla validità del formato: un esempio numerico presente nel prompt veniva copiato alla lettera dai modelli ed è stato quindi sostituito da segnaposto. L'interpretazione dell'output è permissiva, a favore dei giudici: le voci complete di un oggetto troncato vengono conservate e un'etichetta mancante viene ricavata dalle probabilità. La decodifica non è vincolata da una grammatica [46], così che il protocollo resti quello degli altri giudici; questa scelta incide soprattutto sugli errori di formato del modello più piccolo.

Readout contro generazione (B2). I modelli GroundOne addestrati sono eseguiti con lo stesso prompt e lo stesso singolo passaggio, ma il verdetto in corrispondenza di ciascun marcatore è il token con la probabilità più alta sull'intero vocabolario, ossia ciò che produrrebbe la generazione greedy di un token. Le tre lettere, con o senza spazio iniziale, corrispondono alle tre classi, e qualsiasi altro token è un errore di formato. Questo controllo mantiene identici pesi, prompt e costo, e cambia soltanto il fatto che la decisione sia o meno ristretta alle opzioni ammissibili.

Un System One generalista (B3). Il progetto aperto Rizzo-Flow [8] realizza l'idea di System One con un modello da 4B addestrato su decisioni tipizzate generiche. Gira in un proprio server, basato su llama.cpp [2], e viene interrogato via HTTP con il documento e la risposta numerata come stato condiviso e una domanda a scelta a tre classi per frase. Si impiega un modello quantizzato a 4 bit perché quello a 8 bit non trova posto in memoria accanto al sistema operativo.

Un modello decisionale generalista della stessa dimensione (B3b). d1-3B di Liquid AI [26] è un modello decisionale aperto da 3.1B parametri, post-addestrato per decisioni tipizzate su un'ampia miscela di compiti, che legge lettere di risposta ristrette con lo stato condiviso tra le domande: appartiene alla stessa famiglia di readout di GroundOne e ha la dimensione del suo modello più grande, ma è generalista. Permette quindi di isolare il valore della specializzazione rispetto a un modello decisionale già addestrato a decidere, anziché rispetto a uno non addestrato. Viene eseguito zero-shot con il proprio codice di inferenza, in PyTorch sulla GPU

del portatile e senza accesso alla rete, con lo stesso stato e le stesse domande di Rizzo-Flow e al massimo sei domande per chiamata sullo stesso stato; su LLM-AggreFact i suoi documenti sono suddivisi esattamente come quelli di GroundOne e si conserva il massimo tra i frammenti. Un dettaglio pratico è rilevante per la riproducibilità: PyTorch conserva kernel compilati per ogni nuova lunghezza di input sulla GPU Apple e non rilascia quella memoria, perciò la valutazione si svolge in processi worker che vengono riavviati quando la memoria usata dal driver della GPU supera 11 GB, il che lascia invariati gli output.

Encoder specializzati (B4). MiniCheck-RoBERTa [39] e HHEM-2.1-open [40] sono i verificatori di piccole dimensioni più forti disponibili. Valutano ciascuna coppia (documento, frase) in modo indipendente e restituiscono una sola probabilità di supporto; i documenti lunghi sono suddivisi in frammenti di circa 400 parole e si conserva il massimo tra i frammenti, come in MiniCheck. Non disponendo di un output a tre classi, sono confrontati soltanto sulla decisione binaria. Girano con PyTorch sulla GPU del portatile.

Risultati pubblicati. Per LLM-AggreFact, i punteggi di sistemi più grandi tratti dalla classifica pubblica, come Qwen2.5-7B usato zero-shot, sono riportati così come pubblicati e non vengono rieseguiti.

I sistemi più lenti sono valutati su sottocampioni stratificati degli insiemi di test: Rizzo-Flow su 300 risposte di T1, 600 coppie di T3 e 1,100 elementi di T2, di cui ne ha valutati 1,093 (sette superavano il suo contesto); d1-3B su un sottocampione casuale di 4,000 elementi di T2; il 3B non addestrato su 600 risposte di T1; i giudici generativi, infine, sulle 200 risposte del protocollo di latenza descritto più avanti. GroundOne, il 0.5B non addestrato, gli encoder e, su T1 e T3, d1-3B sono valutati sugli insiemi di test completi, e i confronti con un sistema sottocampionato sono sempre condotti sugli elementi valutati da entrambi i sistemi.

5.3 Metriche

Poiché la quota di frasi allucinate differisce tra l'insieme di sviluppo e T1 (Sezione 4.6), le metriche primarie non dipendono dal tasso di base. Per la decisione a tre classi la metrica principale è l'accuratezza bilanciata, cioè la recall media sulle tre classi, affiancata dalla recall di ciascuna classe e dalla quota di ciascuna classe predetta, che rivela un modello in collasso su un'unica risposta. Per la decisione binaria le metriche sono l'accuratezza bilanciata con soglia 0.5, come nella classifica di LLM-AggreFact, e l'AUROC di $P(\text{supported})$, che non dipende da alcuna soglia. La calibrazione è misurata con l'ECE definito nella Sezione 3.4, con il Brier score e con la log-verosimiglianza negativa, mentre la predizione selettiva è valutata

mediante curve rischio–copertura e l'area sottesa a tali curve (AURC) [54]. Tutte le metriche su T1 sono riportate anche per compito e per fascia di K.

5.4 Protocollo statistico

Ogni differenza tra due sistemi è accompagnata da un intervallo di confidenza al 95 % ottenuto con un bootstrap appaiato con 2,000 ricampionamenti [77, 78, 79]. L'unità di ricampionamento non è la frase ma il gruppo usato dal controllo di leakage: il documento sorgente in RAGTruth, il documento in LLM-AggreFact e la pagina di Wikipedia in VitaminC. Le frasi di una stessa risposta, così come le risposte relative a uno stesso documento, sono fortemente correlate, e ricampionarle in modo indipendente produrrebbe intervalli troppo stretti. Su LLM-AggreFact i gruppi sono ricampionati all'interno di ciascuno degli undici dataset, perché la media macro attribuisce a ogni dataset lo stesso peso. Entrambi i sistemi di un confronto sono valutati sugli stessi ricampionamenti, e il p-value riportato è la quota bilaterale di ricampionamenti in cui la differenza cambia segno.

Due limiti di questo protocollo vengono dichiarati in anticipo. In primo luogo, si sottopongono a test circa cinquanta confronti senza alcuna correzione per confronti multipli; le conclusioni poggiano quindi su intervalli ben lontani dallo zero, e non si rivendica alcuna differenza con p-value vicino a 0.05. In secondo luogo, il ricampionamento dei dati non cattura la variabilità dovuta all'addestramento: questa informazione è fornita separatamente dalle due esecuzioni del 0.5B (Sezione 6.2).

5.5 Latenza, energia e hardware

La latenza è misurata su 200 risposte di T1, 50 per ciascuna fascia di K, così che le risposte lunghe siano rappresentate quanto quelle brevi. GroundOne, il readout con una domanda per frase e tutti i giudici generativi girano nel processo stesso su MLX, una risposta alla volta, come farebbe un verificatore all'interno di un assistente locale. Le prime tre risposte servono da riscaldamento (warm-up) e sono escluse. Per le varianti System One ogni risposta è cronometrata tre volte e si conserva la mediana; i giudici generativi sono cronometrati una sola volta, poiché la decodifica greedy è deterministica e ogni esecuzione è molto più lunga. Il tempo riportato è il tempo reale (wall-clock) della chiamata al modello per una risposta, da cui si ricava il tempo per decisione.

Sono stati registrati tempi anche per gli encoder, per Rizzo-Flow e per d1-3B, ma in condizioni che non consentono un confronto equo. Gli encoder giravano con PyTorch e valutavano ogni frase separatamente, rileggendo ogni volta il documento, il che li penalizza; Rizzo-Flow girava in un motore di inferenza diverso, con pesi a 4 bit e dietro un'interfaccia HTTP; d1-3B girava in PyTorch con il proprio codice

e con i riavvii dei worker descritti sopra. Non si avanza quindi alcuna affermazione sulla latenza nei loro confronti.

L'energia è misurata con lo strumento `powermetrics` di macOS, che riporta la potenza di CPU, GPU e Neural Engine ogni 200 millisecondi. Per ciascun sistema, 100 risposte del sottoinsieme di latenza (25 per fascia di K) sono verificate una sola volta, dopo 60 secondi di misura a riposo con il modello già caricato; l'energia di una verifica è la potenza media durante l'esecuzione, al netto della potenza a riposo, moltiplicata per la durata dell'esecuzione e divisa per il numero di risposte. La misura copre le unità di calcolo del chip ma non la memoria, lo schermo o l'archiviazione, ed è stata effettuata di notte con la macchina altrimenti inattiva.

Un solo modello è in esecuzione alla volta, e una guardia di memoria sorveglia ogni job lungo registrando per tutta la sua durata la memoria libera e lo swap. I risultati vengono scritti una sola volta e mai sovrascritti, e ogni esecuzione conserva la propria configurazione, così che ogni numero del Capitolo 6 possa essere ricondotto all'esecuzione che lo ha prodotto.

Capitolo 6

Risultati

Questo capitolo descrive come GroundOne apprende e quanto sia stabile il suo addestramento, che cosa ottiene sui tre insiemi di test congelati, come si colloca rispetto alle baseline e quale contributo fornisca, da sola, la lettura dei logit (readout) del System One. Esamina poi la qualità delle probabilità, il tempo e l'energia necessari per verificare una risposta e gli errori che permangono. Gli intervalli riportati sono intervalli bootstrap a cluster al 95 %, calcolati come descritto nella Sezione 5.4.

6.1 Addestramento

Entrambi i modelli hanno appreso il compito. Prima dell'addestramento, un modello che legge il formato con i marcatori non ha alcuna nozione di ciò che il marcatore richiede, e i suoi verdetti sono al livello del caso: il modello 0.5B risponde "contraddetta" per l'88 % delle frasi di sviluppo e raggiunge un'accuratezza bilanciata a tre classi di 0.32. Nelle esecuzioni completate la loss di addestramento è scesa, entro i primi cento passi, sotto il valore di un modello che avesse appreso soltanto le frequenze delle classi ($\ln 3 \approx 1.10$), e il punteggio sull'insieme di sviluppo è poi cresciuto in modo costante (Figura 6.1). Il modello 0.5B, addestrato su tutte le 13,203 risposte, ha raggiunto il punteggio migliore al passo 600 di 802, con un'accuratezza bilanciata di 0.633 mediata sui tre compiti, per poi restare stabile. Il modello 3B, addestrato per 12.25 ore su 6,500 risposte, ha raggiunto 0.705 al passo 320 di 403 sul proprio sottoinsieme di sviluppo. Misurata sull'intero insieme di sviluppo, l'accuratezza bilanciata a tre classi è di 0.731 per il 0.5B e di 0.784 per il 3B, e in entrambe le esecuzioni tutte e tre le classi sono state predette per l'intera durata dell'addestramento.

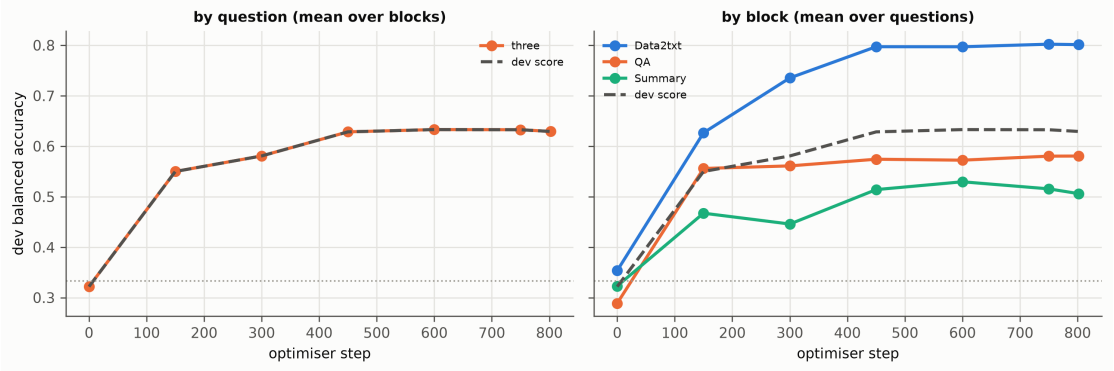


Figura 6.1: Accuratezza bilanciata a tre classi sull'insieme di sviluppo durante l'addestramento del modello 0.5B, complessiva (a sinistra) e per compito (a destra). La linea punteggiata indica il livello del caso. Il 3B mostra lo stesso andamento sul suo sottoinsieme di sviluppo di 200 risposte.

6.2 Variabilità tra seed

La ricetta del modello 0.5B è stata eseguita una seconda volta con un seed diverso, che modifica l'inizializzazione degli adattatori e l'ordine dei batch. Quando l'addestramento converge, converge nello stesso punto (Figura 6.2): la seconda esecuzione raggiunge un'accuratezza bilanciata a tre classi di 0.767 su T1 contro 0.769, un AUROC di 0.907 contro 0.905, 71.5 contro 70.1 su LLM-AggreFact e 0.434 contro 0.442 su VitaminC. Su T1 la differenza appaiata è +0.001 (da −0.008 a +0.011) e non è significativa. La variazione riguarda i dettagli più fini. Gli errori si distribuiscono diversamente tra le classi, con la recall delle frasi supportate e di quelle non menzionate che si sposta di circa 0.04 in direzioni opposte, mentre le porzioni più piccole dei dati oscillano di più: la fascia con quattordici o più decisioni di 0.023, l'AUROC su VitaminC di 0.025 e i singoli dataset di LLM-AggreFact fino a dieci punti, come nel caso di WiCE (67.9 contro 78.0).

Ne discendono due conseguenze. In primo luogo, il bootstrap, che ricampiona i dati di test, non cattura questa fonte di variabilità: su LLM-AggreFact e su VitaminC le due esecuzioni della stessa ricetta differiscono più di quanto i rispettivi intervalli consentano (−0.013 sulla media macro di T2, −0.025 sull'AUROC di T3). Le differenze tra sistemi inferiori alla dispersione tra seed, circa 1.5 punti sulla media macro di LLM-AggreFact e 0.025 di AUROC su VitaminC, non vengono quindi interpretate; lo stesso vale per i singoli dataset di LLM-AggreFact, a meno che lo scarto non superi circa cinque punti.

In secondo luogo, l'addestramento non converge sempre. Un'esecuzione con un terzo seed è rimasta vicina al livello del caso: il suo punteggio di sviluppo era 0.369 al passo 150 e 0.333 al passo 300, quando prediceva "supportata" per ogni

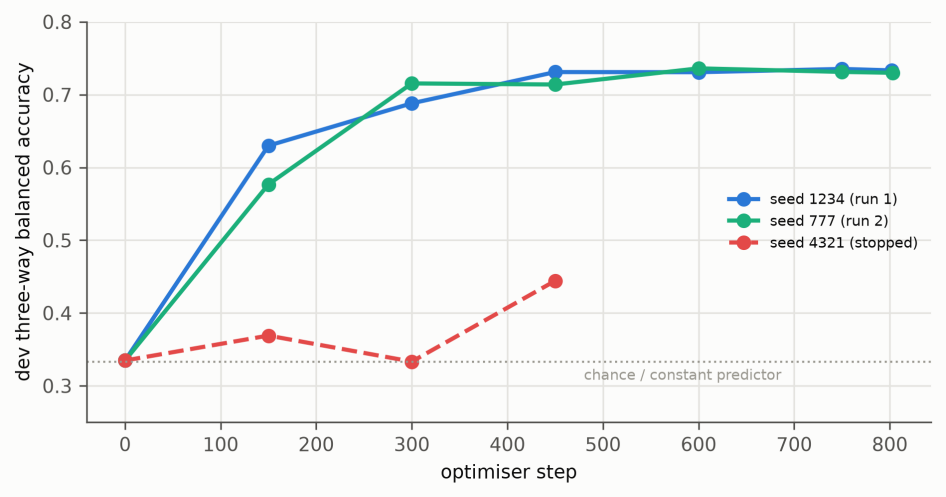


Figura 6.2: Accuratezza bilanciata a tre classi sull'insieme di sviluppo del modello 0.5B durante l'addestramento, con la stessa ricetta e tre seed. Due esecuzioni convergono allo stesso livello; la terza è collassata su un predittore costante ed è stata interrotta.

frase, e la loss di addestramento è rimasta prossima al valore di un modello privo di informazione. L'esecuzione è stata interrotta al passo 470 di 801, quando il punteggio era risalito a 0.444, per cui non è noto se si sarebbe ripresa entro la fine dell'epoca; essendo incompleta, è esclusa dalle tabelle. La ricetta presenta due caratteristiche note per rendere fragile il fine-tuning: i gradienti non sono sottoposti a clipping e l'ottimizzatore opera senza correzione del bias, impostazione predefinita del framework, che è stata individuata come una delle cause principali dei fallimenti nel fine-tuning [60, 59]. Un solo collasso su tre esecuzioni non basta a misurare un tasso di fallimento, ma mostra che la ricetta non è del tutto robusta rispetto all'inizializzazione.

6.3 Accuratezza sull'insieme di test RAG (T1)

Sulle 17,284 frasi di T1 il modello 0.5B raggiunge un'accuratezza bilanciata a tre classi di 0.769 e il modello 3B di 0.832 (Tabella 6.1); la differenza, +0.063 (da +0.052 a +0.076), è l'effetto più netto della dimensione del modello. Sulla decisione binaria il 0.5B raggiunge un'accuratezza bilanciata di 0.815 e un AUROC di 0.905, il 3B rispettivamente 0.874 e 0.940. Entrambi superano ampiamente gli indizi che non verificano mai una frase rispetto al suo documento: la sovrapposizione lessicale raggiunge un AUROC di 0.737 e il classificatore basato sulla sola frase un'accuratezza bilanciata a tre classi di 0.570.

Tabella 6.1: GroundOne su T1, per compito e per numero di decisioni per risposta (K): accuratezza bilanciata a tre classi, recall delle frasi contraddette e AUROC di P(supported).

		GroundOne 0.5B			GroundOne 3B		
Sottoinsieme	Frasi	Acc. bil.	Contr.	AUROC	Acc. bil.	Contr.	AUROC
Tutte	17,284	0.769	0.68	0.905	0.832	0.76	0.940
Risposta a domande	4,876	0.589	0.03	0.916	0.699	0.30	0.948
Riassunto	4,538	0.544	0.13	0.775	0.629	0.30	0.873
Data-to-text	7,870	0.812	0.84	0.929	0.873	0.90	0.955
$K = 1$–4	2,434	0.606	0.18	0.857	0.677	0.25	0.921
$K = 5$–8	8,012	0.758	0.67	0.899	0.829	0.78	0.936
$K = 9$–13	5,201	0.824	0.82	0.926	0.863	0.86	0.945
$K \geq 14$	1,637	0.723	0.50	0.908	0.839	0.68	0.957

Emergono tre osservazioni. Anzitutto, l'accuratezza non peggiora con la lunghezza della risposta: le risposte con quattordici o più decisioni sono tra quelle gestite meglio, con 0.723 per il 0.5B e 0.839 per il 3B, mentre la fascia più difficile è la più corta, che contiene soprattutto risposte di question answering e di riassunto. Il dato è rilevante perché è proprio sulle risposte lunghe che un System One fa risparmiare più tempo; su queste risposte il guadagno del 3B rispetto al 0.5B è +0.116 (da +0.050 a +0.180). In secondo luogo, le contraddizioni vengono riconosciute soprattutto nel data-to-text, dove RAGTruth le concentra. Nel question answering e nel riassunto il 0.5B separa bene le frasi supportate da quelle non menzionate, ma non individua quasi nessuna contraddizione, con una recall di 0.03 e 0.13. Il 3B porta entrambe a 0.30: la dimensione del modello compensa in parte la scarsità di tali esempi nei dati di addestramento. Infine, question answering e riassunto costituiscono la prova più chiara che i modelli leggono il documento: in questi compiti il classificatore basato sulla sola frase resta al livello del caso (0.36 e 0.35), mentre i due modelli raggiungono 0.59–0.70 e 0.54–0.63.

Le stesse predizioni possono essere lette a livello di risposta intera, che è il modo in cui il Decision Index della comunità valuta i modelli decisionali generalisti su RAGTruth: una risposta è considerata allucinata se contiene un qualsiasi contenuto non supportato, e la metrica è l'F1 di questa classe, che un sistema che risponda sempre "allucinata" raggiunge già con 0.52 [28]. Prendendo il minimo della P(supported) calibrata sulle frasi di una risposta, con una soglia stimata sull'insieme di sviluppo, GroundOne raggiunge 0.733 con il 0.5B e 0.784 con il 3B sulle 2,647 risposte di T1. Nella classifica nessun modello decisionale generalista sotto il miliardo di parametri supera 0.55, Jev raggiunge 0.765 e il modello migliore, da 27B parametri, 0.803. Il confronto è soltanto indicativo, perché GroundOne è

stato addestrato su RAGTruth mentre i modelli in classifica sono valutati zero-shot, con un prompt diverso e su 53 risposte in più; colloca tuttavia uno specialista da 0.5B molto al di sopra dei modelli decisionali generalisti della stessa dimensione.

6.4 Confronto con le baseline

La Figura 6.3 confronta tutti i sistemi su T1. Gli encoder specializzati raggiungono un AUROC di 0.814 (MiniCheck-RoBERTa) e 0.821 (HHEM), superiore alla sovrapposizione lessicale ma inferiore a quello di entrambi i modelli GroundOne: il 0.5B li supera di +0.091 (da +0.074 a +0.107) e +0.083 (da +0.066 a +0.099), il 3B di +0.126 e +0.119. Il confronto per compito ridimensiona questo risultato. Il vantaggio di GroundOne deriva in gran parte dal data-to-text, dove è addestrato nel dominio e dove gli encoder sono più deboli (AUROC 0.71–0.75), mentre sul riassunto MiniCheck-RoBERTa è migliore del modello 0.5B (0.872 contro 0.775) e alla pari con il 3B (0.873). I modelli non addestrati mostrano quanta parte dell'accuratezza sia dovuta all'addestramento: il 0.5B è al livello del caso perché non risponde mai "supportata", mentre il 3B, a cui viene posta la stessa domanda, raggiunge 0.437 contro 0.818 dopo l'addestramento, misurati sulle stesse risposte. Quanto contribuisca il readout ristretto, oltre all'addestramento, viene misurato separatamente nella Sezione 6.6.

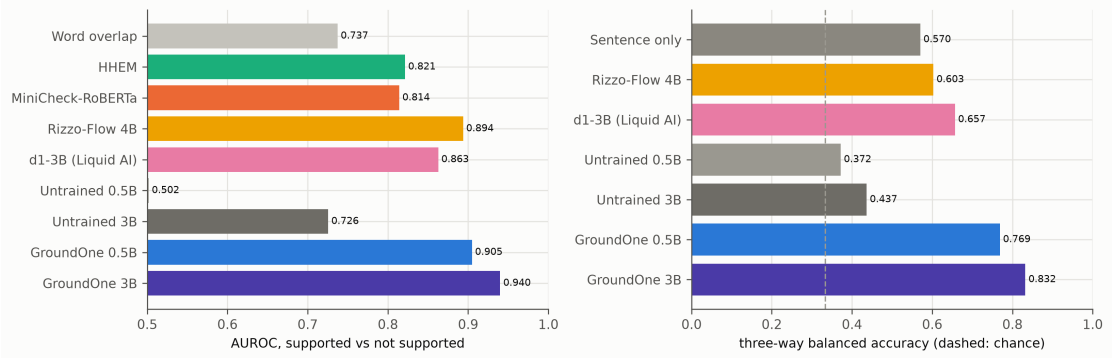


Figura 6.3: T1: AUROC di $P(\text{supported})$ per ogni sistema (a sinistra) e accuratezza bilanciata a tre classi per i sistemi con output a tre classi (a destra). Rizzo-Flow è valutato su 300 risposte e il 3B non addestrato su 600; tutti gli altri sistemi, compreso il modello decisionale generalista d1-3B, sull'intero insieme di test.

Il confronto più istruttivo è quello con il System One generalista. Sulle stesse 300 risposte di T1, Rizzo-Flow, un modello da 4B sottoposto a fine-tuning per decisioni generiche, raggiunge un'accuratezza bilanciata a tre classi di 0.603, contro 0.798 del GroundOne 0.5B e 0.840 del 3B; le differenze, +0.195 (da +0.136 a +0.249)

e +0.237 (da +0.184 a +0.285), sono ampie. Il suo AUROC (0.894) è vicino a quello del 0.5B su queste risposte (0.916; la differenza non è significativa) perché ordina bene le frasi ma predice "supportata" per il 90 % di esse; il 3B le ordina meglio (+0.056, da +0.034 a +0.079). Uno specialista otto volte più piccolo supera dunque il generalista nella decisione per cui è stato costruito, eguagliandolo nella capacità di ordinare le frasi.

Un modello decisionale generalista della stessa dimensione offre la verifica più netta della specializzazione, poiché d1-3B è già stato addestrato a prendere decisioni tipizzate e si differenzia da GroundOne soprattutto per i dati di addestramento. Sull'intero T1 raggiunge un'accuratezza bilanciata a tre classi di 0.657 e un AUROC di 0.863. Entrambi i modelli GroundOne sono nettamente migliori: il 0.5B, sei volte più piccolo, di +0.112 (da +0.092 a +0.133) nell'accuratezza a tre classi e di +0.042 in AUROC, il 3B di +0.175 (da +0.157 a +0.195) e +0.077. Lo scarto è massimo sulle risposte lunghe, dove d1-3B scende a 0.527 con quattordici o più decisioni mentre il GroundOne 3B resta a 0.839 (+0.312, da +0.230 a +0.405), e d1-3B riconosce soltanto la metà delle frasi non menzionate (recall 0.49). Nel dominio delle risposte generate con il recupero, la specializzazione vale dunque molto più di un addestramento generale alle decisioni.

Su LLM-AggreFact (T2) il modello 0.5B raggiunge un'accuratezza bilanciata media macro di 70.1 sugli undici dataset, contro 64.2 dello stesso modello usato zero-shot e 72.8 del modello 7B zero-shot nella classifica pubblica. Resta al di sotto di MiniCheck-RoBERTa (74.0) di 3.9 punti, una differenza significativa. Sui quattro dataset più vicini alla generazione aumentata dal recupero i due sono alla pari (74.4 contro 74.6), e nella colonna di RAGTruth GroundOne è in vantaggio, 82.1 contro 76.5, un confronto che lo favorisce perché è stato addestrato sulle risposte di addestramento di RAGTruth. Al di fuori della generazione aumentata dal recupero il 0.5B è inferiore all'encoder di sette-nove punti su AggreFact-CNN, AggreFact-XSum, FactCheck-GPT e REVEAL, il costo atteso di un addestramento sui soli dati RAG.

Il modello 3B, valutato sull'intero insieme di test, colma gran parte di questo divario. La sua accuratezza bilanciata media macro è 74.8, alla pari con MiniCheck-RoBERTa (74.0; differenza appaiata +0.8 punti, intervallo da −0.5 a +2.1) e superiore al punteggio zero-shot pubblicato per il modello 7B (72.8, per il quale non è possibile calcolare un intervallo). Sui dataset RAG raggiunge 76.9, 2.4 punti sopra l'encoder (intervallo da +1.4 a +3.3). Il suo guadagno rispetto al 0.5B, +4.7 punti (da +3.4 a +5.9), è diffuso, con i miglioramenti maggiori su REVEAL, WiCE, FactCheck-GPT e XSum: la dimensione del modello migliora quindi il trasferimento oltre il dominio di addestramento. Il 3B resta nettamente al di sotto dell'encoder soltanto sul riassunto di notizie (AggreFact-CNN, 59.3 contro 67.1). Rizzo-Flow, sui 1,093 elementi del suo sottocampione, raggiunge 61.5; su di essi il GroundOne 3B è migliore di 12.0 punti (da +7.4 a +16.5). Una valutazione indipendente del System

One chiuso Jev, sullo stesso insieme di test e con la stessa metrica, riporta 78.6, e 76.9 per un modello aperto da 27B letto attraverso le lettere di risposta, con prompt diversi da quelli usati qui, per cui non è possibile calcolare alcun intervallo [21]. Su un benchmark ampio di groundedness, dunque, uno specialista da 3B addestrato soltanto su risposte RAG eguaglia il miglior encoder di piccole dimensioni, ma resta al di sotto dello stato dell'arte chiuso e di modelli aperti molto più grandi. Il generalista d1-3B conferma il quadro a parità di dimensione: su un sottocampione casuale di 4,000 elementi raggiunge 75.7, contro 74.3 del GroundOne 3B sugli stessi elementi, una differenza non significativa (-1.3, da -4.0 a $+1.0$), mentre supera il 0.5B di 6.5 punti. È più debole nella colonna di RAGTruth (78.9 contro 87.3) e nettamente più forte su WiCE e sul riassunto di dialoghi.

6.5 Trasferimento a un dominio diverso (T3)

Su VitaminC il quadro si capovolge. Sull'intero insieme di test il 0.5B raggiunge un'accuratezza bilanciata a tre classi di 0.442 e il 3B di 0.527, con AUROC di 0.722 e 0.817, mentre MiniCheck-RoBERTa raggiunge un AUROC di 0.968. Sulle 600 coppie valutate con Rizzo-Flow, il modello generalista raggiunge 0.712 e un AUROC di 0.909, contro 0.442 e 0.744 del 0.5B e 0.537 e 0.845 del 3B. Due fatti aiutano a contestualizzare questi numeri. MiniCheck-RoBERTa parte dal modello AlignScore, i cui dati di addestramento includono VitaminC [38], per cui T3 appartiene alla sua distribuzione di addestramento, mentre per GroundOne misura il trasferimento da RAGTruth. Inoltre, il guadagno del 3B rispetto al 0.5B su T3 deriva dalla dimensione del modello più che dall'addestramento su dati RAG: il 3B non addestrato raggiunge già 0.531 e un AUROC di 0.821. Il modello decisionale generalista della stessa dimensione rende esplicito il contrasto: d1-3B raggiunge 0.724 e un AUROC di 0.913 sull'intero T3, davanti al GroundOne 3B di 0.197 e 0.096, l'opposto di quanto accade su T1. La specializzazione sulle risposte RAG migliora quindi in modo sostanziale il compito RAG, ma non produce, da sola, un verificatore di fatti generale. La Sezione 6.11 mostra quali casi vengono mancati.

6.6 Readout e generazione a confronto

Il controllo B2 isola il readout. Con gli stessi pesi ottenuti dal fine-tuning, lo stesso prompt e lo stesso singolo passaggio, il verdetto a ciascun marcatore viene letto dalle tre lettere di risposta, come fa GroundOne, oppure preso come il token che la generazione greedy emetterebbe (Figura 6.4). Per il 0.5B il token generato produce un'accuratezza bilanciata a tre classi di 0.521 contro 0.769 del readout, una differenza di $+0.248$ (da $+0.228$ a $+0.269$): in un quarto dei marcatori (24.7 %)

il modello non emetterebbe affatto una lettera, ma più spesso un semplice spazio, l'inizio di una risposta più lunga. Per il 3B il token generato è quasi sempre una lettera (1.2 % di fallimenti) e concorda con il readout nel 92 % dei marcatori, eppure il readout resta migliore, 0.832 contro 0.761 (+0.071, da +0.059 a +0.083). Sulle risposte con quattordici o più decisioni lo scarto è massimo per il modello piccolo, +0.346.

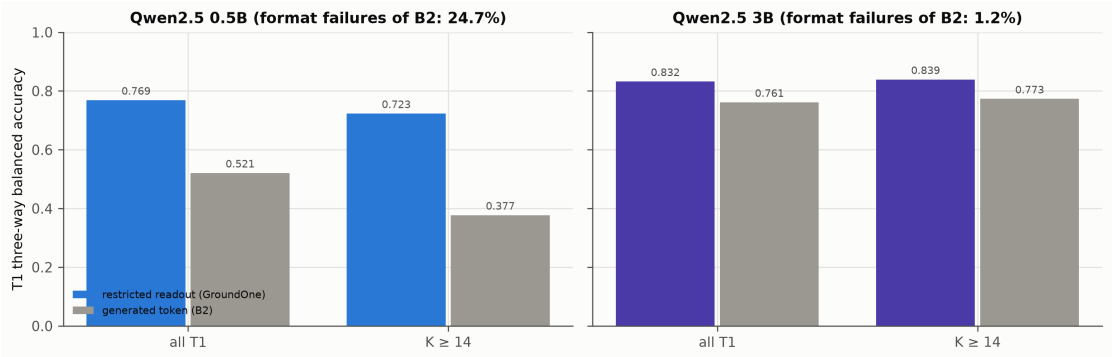


Figura 6.4: Controllo B2 sull'intero T1: con gli stessi pesi e lo stesso passaggio in avanti, il verdetto viene letto dalle tre lettere di risposta (GroundOne) oppure preso come il token che la generazione greedy emetterebbe (i fallimenti di formato sono contati come errori).

La latenza delle due opzioni è la stessa, poiché entrambe usano un solo passaggio; ciò che cambia è la regola di decisione. Restringere la scelta alle opzioni ammissibili elimina per costruzione i fallimenti di formato e, dato che il modello è addestrato soltanto a ordinare le tre lettere, fornisce anche la decisione più accurata (Sezione 3.2). Quanto più il modello è piccolo, tanto più questa restrizione conta. L'aspettativa, spesso ripetuta, che leggere i logit di risposta e generare un singolo token di risposta siano equivalenti vale quindi per il costo, ma non per l'accuratezza di un verificatore sottoposto a fine-tuning.

6.7 Calibrazione e predizione selettiva

Le probabilità grezze sottostimano il supporto. Nel diagramma di affidabilità (Figura 6.5) la curva grezza si trova sopra la diagonale, perché i pesi delle classi addestrano il modello come se le classi fossero bilanciate; l'ECE binario su T1 è 0.30 per il 0.5B e 0.18 per il 3B. La temperatura e il bias stimati sull'insieme di sviluppo portano l'ECE su T1 a 0.019 per entrambi i modelli, e a 0.020 per la seconda esecuzione del 0.5B, e riducono la log-verosimiglianza negativa da 0.55 a 0.21 (0.5B) e da 0.36 a 0.17 (3B), lasciando l'AUROC praticamente invariato. La

mappa stimata sui dati di sviluppo si trasferisce quindi all'insieme di test dello stesso dominio.

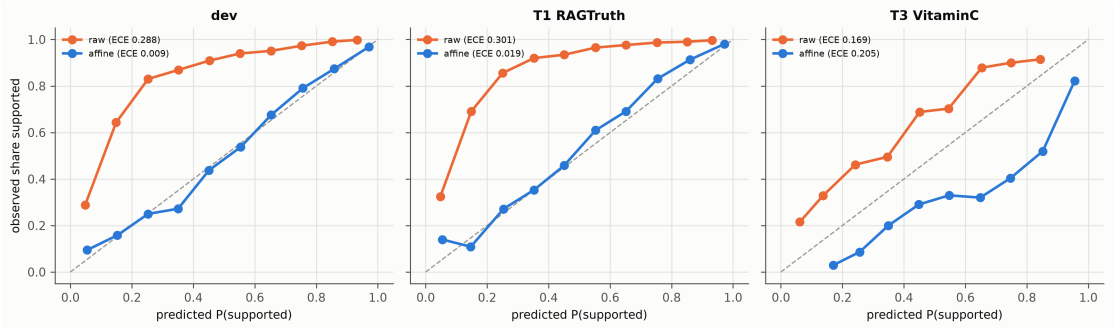


Figura 6.5: Affidabilità di $P(\text{supported})$ del 0.5B prima (grezza) e dopo la calibrazione sull'insieme di sviluppo, per l'insieme di sviluppo, T1 e T3. I punti sono intervalli con almeno 20 frasi.

La calibrazione ha una conseguenza di cui occorre tenere conto. Il bias ripristina la frequenza reale di ciascuna classe, cosicché la classe calibrata più probabile diventa più spesso "supportata": per il 0.5B su T1 l'accuratezza sale da 0.82 a 0.93, ma l'accuratezza bilanciata a tre classi scende da 0.77 a 0.63. Scegliere la classe calibrata più probabile non è quindi una buona regola di decisione quando i tre tipi di errore hanno la stessa importanza. La regola di servizio della Sezione 3.4 risolve il problema: con la soglia stimata sui dati di sviluppo, $\tau = 0.935$ per il 0.5B, 0.895 per la sua seconda esecuzione e 0.925 per il 3B, raggiunge 0.758, 0.769 e 0.830 su T1, entro 0.01 dalla decisione grezza valutata in tutto il capitolo, mantenendo coerenti le etichette e le probabilità calibrate. Le soglie sono elevate perché il 91 % delle frasi di RAGTruth è supportato: una frase viene restituita come supportata soltanto quando il modello è molto sicuro.

Usate come confidenze, le probabilità calibrate rendono efficace la predizione selettiva: rinviare le frasi meno sicure abbassa rapidamente il tasso di errore della decisione binaria, con un'area sotto la curva rischio–copertura di 0.015 (0.5B) e 0.010 (3B) su T1 (Figura 6.6).

La calibrazione non si trasferisce a VitaminC, dove l'ECE binario sale da 0.17 a 0.21 per il 0.5B e da 0.14 a 0.19 per il 3B, e dove la regola di servizio perde 0.02–0.04 rispetto alla decisione grezza. Il bias codifica la frequenza di base di RAGTruth, mentre VitaminC è bilanciato per costruzione. Una calibrazione è valida per la distribuzione su cui è stata stimata, per cui un impiego reale richiede un piccolo campione etichettato dei propri dati.

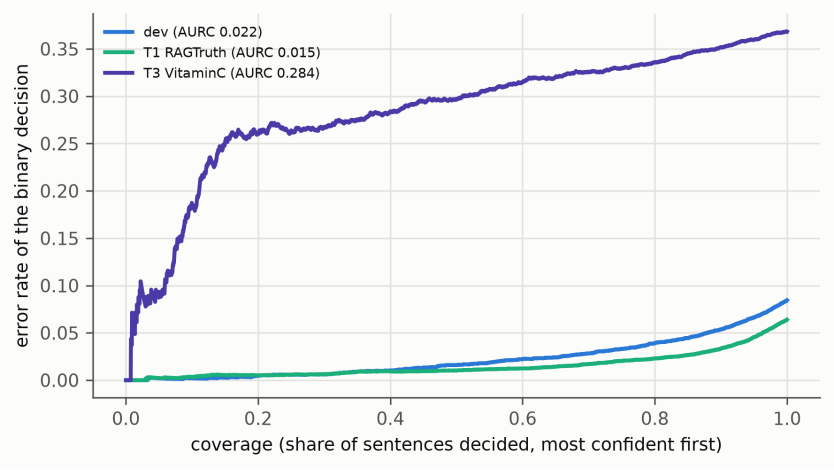


Figura 6.6: Curve rischio–copertura della decisione binaria del 0.5B con probabilità calibrate: tasso di errore man mano che le frasi meno sicure vengono rinviate.

6.8 Probabilità a parità di output

I giudici a parità di output rispondono alla domanda se un giudice generativo della stessa dimensione, a cui si chiede lo stesso contenuto di GroundOne, possa eguagliare la qualità delle sue probabilità. Non può farlo (Figura 6.7). Sulle 200 risposte del sottoinsieme di latenza, con tutti i sistemi calibrati allo stesso modo sui dati di sviluppo, l'AUROC di $P(\text{supported})$ è 0.94 e 0.97 per GroundOne, 0.48 e 0.68 per il giudice le cui probabilità sono lette dai logit e 0.47 e 0.58 per il giudice che le scrive, rispettivamente per il 0.5B e per il 3B. L'area sotto la curva rischio–copertura è 0.009 e 0.005 per GroundOne contro valori da 0.048 a 0.063 per i giudici, da cinque a dodici volte maggiori. L'accuratezza delle decisioni segue lo stesso andamento: 0.835 e 0.900 per GroundOne contro al massimo 0.41 per qualsiasi giudice.

Due ulteriori osservazioni sono rilevanti per la pratica. Le probabilità scritte a parole non portano quasi alcuna informazione: il 3B scrive un vettore con tutta la massa su una sola classe per il 93 % delle frasi, usando soltanto sedici vettori distinti, e il 0.5B scrive parità non informative, come probabilità uguali per tutte le classi; contando come fallimenti le parità prive di etichetta, la sua accuratezza bilanciata scende da 0.234 a 0.081. Ciò concorda con l'osservazione che la confidenza verbalizzata dei modelli piccoli è grossolana e poco informativa [53, 52]. In secondo luogo, l'ECE da solo nasconderebbe tutto questo: dopo la calibrazione ogni sistema, giudici compresi, raggiunge un ECE binario compreso tra 0.03 e 0.05, perché una mappa di calibrazione può allineare la confidenza media alla frequenza di base anche quando le probabilità non separano le classi. La capacità discriminativa va

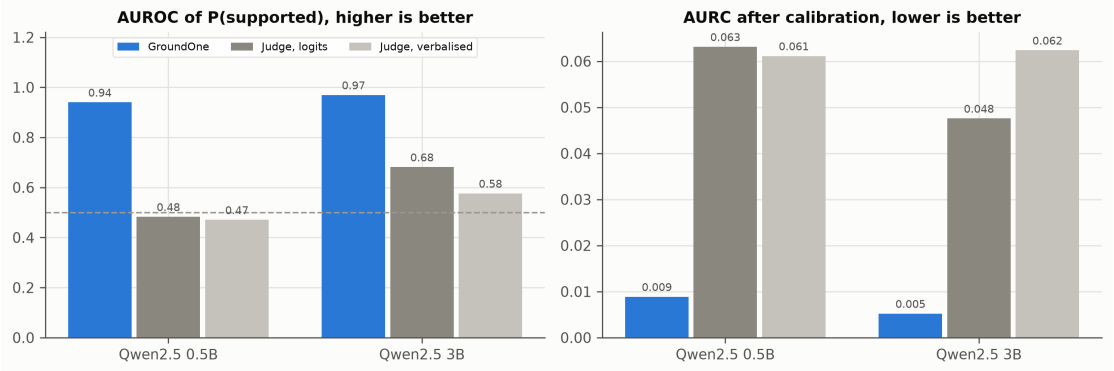


Figura 6.7: Confronto a parità di output sulle 200 risposte di latenza: giudici generativi della stessa dimensione a cui si chiede l'output di GroundOne, con probabilità lette dai logit della lettera generata oppure scritte in JSON; tutti i sistemi sono calibrati sui dati di sviluppo con la stessa mappa. Solo frasi con un verdetto valido.

misurata separatamente, con l'AUROC e la curva rischio–copertura.

Chiedere le probabilità ha inoltre un costo. Il giudice che le legge dai logit è veloce quanto il giudice sintetico, poiché genera le stesse lettere, mentre il giudice che scrive l'oggetto JSON richiede 17.0 secondi per risposta con quattordici o più decisioni sul 3B, tanto quanto il giudice verboso, per un'accuratezza non migliore di quella del giudice sintetico.

6.9 Latenza in funzione di K

La Figura 6.8 mostra la misura centrale della tesi: il tempo necessario per verificare una risposta, sulle stesse 200 risposte di T1, al crescere del numero di decisioni. Tra i sistemi eseguiti in-process su MLX, GroundOne è il più veloce a ogni dimensione e in ogni fascia di K, e il suo vantaggio cresce con K. Per le risposte con quattordici o più decisioni il GroundOne 0.5B impiega 0.41 secondi, contro 0.80 del giudice sintetico e 2.51 del giudice verboso della stessa dimensione: è 1.95 volte più veloce del primo (da 1.82 a 2.14) e 6.16 volte più veloce del secondo (da 5.72 a 6.67). Il GroundOne 3B impiega 2.57 secondi, contro 4.44 e 15.86, ossia è 1.73 volte (da 1.65 a 1.82) e 6.18 volte (da 5.88 a 6.65) più veloce. Leggere tutti i verdetti in un solo passaggio è anche più rapido che porre una domanda per frase in un batch, di un fattore 1.4–1.5 ai valori più alti di K. La dimensione del modello è l'altra leva: il GroundOne 0.5B verifica una risposta circa sei volte più rapidamente del 3B (0.33 contro 1.99 secondi in mediana).

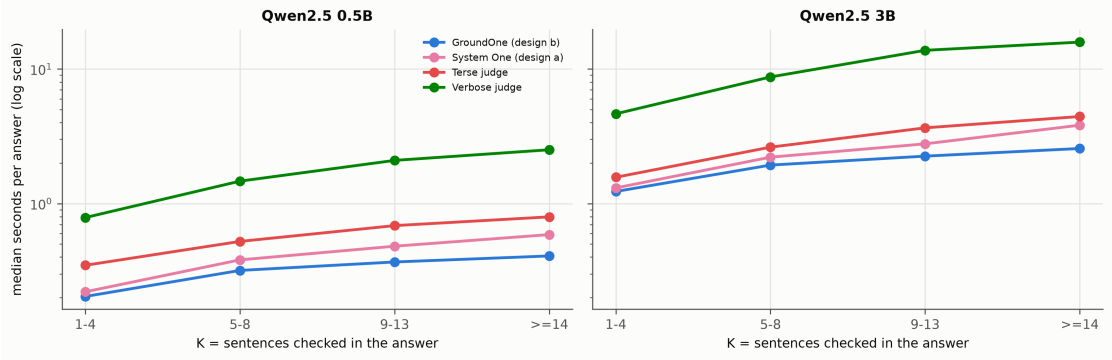


Figura 6.8: Tempo mediano per risposta in funzione di K sulle stesse 200 risposte di T1 (MLX in-process, una risposta alla volta, M4 con 16 GB), per GroundOne, il System One con una domanda per frase e i giudici generativi sintetico e verboso. Scala logaritmica.

La velocità non è l'unica differenza (Figura 6.9). I giudici generativi non addestrati sono anche molto meno accurati e non sempre rispettano il formato di output. Il giudice verboso 0.5B produce un verdetto interpretabile soltanto per il 3 % delle frasi, perché per lo più ricopia la risposta, e il giudice sintetico 0.5B fallisce nel 23 % dei casi. I giudici 3B rispettano il formato molto meglio (0.1 % e 7.6 % di fallimenti), ma raggiungono un'accuratezza bilanciata a tre classi di appena 0.40 circa, contro 0.90 del GroundOne 3B sulle stesse risposte. Il giudice verboso 3B scrive in media 248 token per risposta, che devono essere tutti generati prima che il verdetto sia utilizzabile, mentre GroundOne non ne genera alcuno.

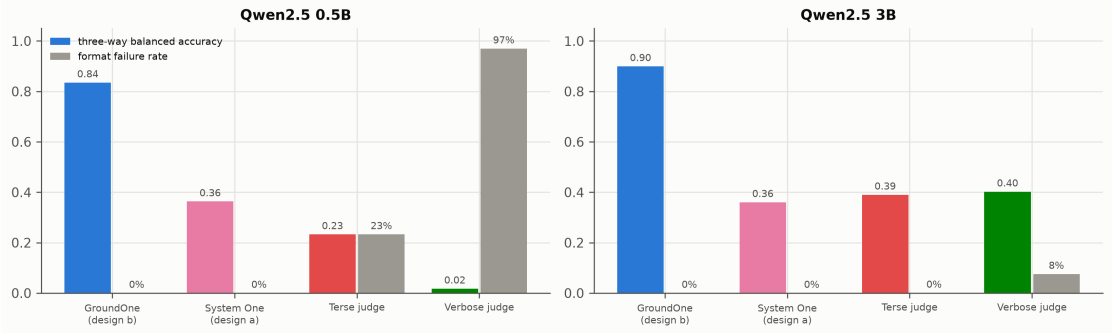


Figura 6.9: Sul sottoinsieme di latenza: accuratezza bilanciata a tre classi (i fallimenti di formato sono contati come errori) e quota di frasi il cui verdetto non è stato possibile interpretare.

6.10 Energia per verifica

La misura dell'energia aggiunge una precisazione che la sola latenza nasconde (Figura 6.10). GroundOne consuma meno energia a entrambe le dimensioni: 5.5 joule per risposta per il 0.5B e 33 per il 3B, contro 8.9 e 43 del giudice sintetico e 22.5 e 112 del giudice verboso. Il suo vantaggio, tuttavia, è minore di quello in termini di tempo: 1.6 e 1.3 volte meno energia del giudice sintetico, e 4.1 e 3.4 volte meno di quello verboso. La ragione sta nella potenza assorbita durante il calcolo. GroundOne trascorre il suo tempo in un unico, lungo prefill, che mantiene la GPU occupata e assorbe circa 16 W oltre il consumo a riposo, mentre la decodifica autoregressiva è limitata dalla banda di memoria e assorbe 11–14 W, ma per un tempo molto più lungo. È il comportamento atteso dalle due fasi dell'inferenza dei transformer [10], e implica che un'accelerazione non vada mai letta come un risparmio di energia equivalente [16, 17].

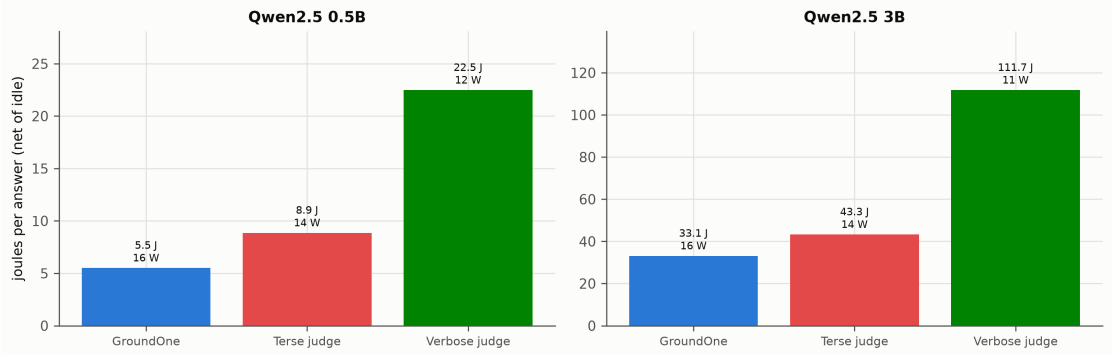


Figura 6.10: Energia per risposta verificata su 100 risposte di latenza (powermetrics, CPU, GPU e Neural Engine, al netto della potenza a riposo), con la potenza netta media durante l'esecuzione.

In termini assoluti i valori sono piccoli: 5.5 joule corrispondono a circa 0.0015 wattora, per cui il GroundOne 0.5B potrebbe verificare all'incirca seicento risposte con l'energia di un wattora, interamente sul portatile.

6.11 Analisi degli errori

Il risultato debole su VitaminC e le contraddizioni mancate su T1 hanno una causa comune, che la sovrapposizione lessicale mette in luce. Su VitaminC, quando un'affermazione supportata condivide poche parole con la sua evidenza (sovrapposizione inferiore a 0.4), il 0.5B la riconosce soltanto nel 2 % dei casi, contro il 64 % quando la sovrapposizione supera 0.7, e non riconosce mai una contraddizione in un'affermazione priva di cifre. I modelli si basano in parte sulla sovrapposizione

lessicale: un'affermazione parafrasata appare "non menzionata", e una piccola modifica a una frase copiata, spesso un numero, passa inosservata.

La stessa debolezza è presente su T1, con un peso minore. Quando una frase condivide poche o alcune parole con il documento, l'accuratezza bilanciata a tre classi del 0.5B è 0.77, ma per le frasi che copiano dal documento più del 70 % delle proprie parole scende a 0.54, con una recall di 0.24 per le frasi contraddette e di 0.43 per quelle non menzionate. Il 3B riduce questo punto cieco senza eliminarlo: sulle stesse frasi la sua recall è 0.36 e 0.56, e la sua accuratezza bilanciata 0.63. Le allucinazioni nascoste in frasi che per il resto copiano la fonte rimangono il limite principale di entrambi i modelli. VitaminC non è più difficile in termini assoluti: è composto per lo più proprio da casi di questo tipo, supporto parafrasato e modifiche minime, che RAGTruth contiene in numero minore. L'Appendice A mostra l'output di ogni sistema sulla stessa risposta.

Capitolo 7

Discussione, limiti e minacce alla validità

Questo capitolo interpreta i risultati alla luce delle cinque domande di ricerca, per poi indicare ciò che gli esperimenti non sono in grado di dimostrare e quali scelte avrebbero potuto distorcerli.

7.1 Quando conviene un System One specialista

Specializzazione (RQ1). È l'addestramento sulla decisione a rendere GroundOne accurato. I modelli non addestrati si collocano vicino al caso, mentre gli stessi modelli dopo l'addestramento raggiungono 0.769 e 0.832 di accuratezza bilanciata a tre classi sull'insieme di test di RAGTruth, superando gli encoder specializzati in AUROC e un System One generalista otto volte più grande nell'accuratezza a tre classi. Il vantaggio si concentra là dove si trovano i dati di addestramento: su LLM-AggreFact il 3B è alla pari con il miglior encoder di piccole dimensioni, anziché superarlo, e resta al di sotto del System One chiuso Jev e di un modello aperto da 27B secondo misure indipendenti; il 0.5B rimane sotto l'encoder, e su VitaminC il 3B addestrato non fa meglio di quello non addestrato. Il confronto con un modello decisionale generalista della stessa dimensione rende esplicito il compromesso. d1-3B, già addestrato a decidere ma su un'ampia miscela di compiti, è nettamente dietro GroundOne su RAGTruth, soprattutto sulle risposte lunghe, alla pari con il 3B su LLM-AggreFact e nettamente avanti su VitaminC. Uno specialista è quindi una scelta solida per il dominio su cui è stato addestrato e un debole verificatore di fatti generale: la specializzazione garantisce accuratezza e robustezza sulle risposte lunghe nel proprio dominio al prezzo del trasferimento, mentre ciò che migliora il trasferimento al di fuori di esso è la dimensione del modello, oppure un'ampia miscela di addestramento.

Readout contro generazione (RQ2). Il meccanismo System One contribuisce in due modi distinti. Il primo è il costo: un solo passaggio sulla risposta sostituisce la generazione di K verdetti, per cui il tempo risparmiato cresce con la lunghezza della risposta, fino a un fattore di circa due rispetto a un giudice sintetico e di sei rispetto a uno verboso per risposte di quattordici o più frasi. Il secondo, che questa tesi non si attendeva, è l'accuratezza. A parità di pesi e di costo, decidere tra le tre lettere ammissibili è più accurato che prendere il token che il modello genererebbe, di 0.25 per il 0.5B e di 0.07 per il 3B. Un System One addestrato va dunque inteso più come un classificatore a tre classi posto sopra un modello linguistico che come un generatore che si arresta dopo un token, e questa distinzione conta soprattutto per i modelli piccoli, che sono quelli alla portata di un portatile. Il confronto con i giudici generativi, infine, misura anche l'addestramento: i giudici non sono addestrati, perciò le ampie differenze di accuratezza tra essi e GroundOne vanno lette come l'effetto combinato della specializzazione e del readout (la lettura dei logit).

Calibrazione (RQ3). Una temperatura e un bias stimati sui dati di sviluppo fanno sì che le probabilità corrispondano alle frequenze osservate sull'insieme di test dello stesso dominio, e le probabilità calibrate rendono efficace la predizione selettiva. Due lezioni vanno oltre GroundOne. In primo luogo, l'errore di calibrazione atteso, da solo, non basta a ordinare i sistemi: dopo la stessa calibrazione, giudici le cui probabilità separano a malapena le classi raggiungono lo stesso ECE di GroundOne, e soltanto l'AUROC e la curva rischio–copertura rivelano la differenza. In secondo luogo, decisioni e probabilità devono essere ricavate in modo coerente: la classe calibrata più probabile collassa verso la classe maggioritaria, mentre una soglia stimata sui dati di sviluppo recupera la decisione bilanciata e mantiene l'etichetta coerente con la probabilità mostrata all'utente. La calibrazione è valida soltanto per la distribuzione su cui è stata stimata, come mostrano i risultati su VitaminC.

Costo sul dispositivo (RQ4). Il verificatore trova agevolmente posto nel portatile da 16 GB: l'addestramento raggiunge picchi di 2.6 e 5.9 GB, e la verifica di una risposta lunga richiede 0.41 secondi con il 0.5B e 2.57 con il 3B, con un'energia di 5.5 e 33 joule. Il 0.5B rispetta un budget interattivo; il 3B, per una risposta tipica, è più lento di un obiettivo interattivo e scambia tempo con accuratezza. Le misure di energia precisano i risultati sulla velocità: un singolo prefill lungo assorbe più potenza della decodifica token per token, sicché l'energia risparmiata è inferiore al tempo risparmiato, e la velocità non va interpretata come un risparmio equivalente di batteria. La memoria necessaria quando il verificatore gira accanto a un generatore e a un retriever non è stata misurata.

Oltre un punteggio binario (RQ5). L'output a tre classi separa bene le frasi contraddette da quelle non menzionate nel data-to-text, dove RAGTruth concentra le proprie contraddizioni, e molto meno nel question answering e nella sintesi, dove le contraddizioni sono rare nei dati di addestramento. Il 3B migliora considerevolmente questo aspetto, il che suggerisce che al modello piccolo manchino contraddizioni più varie in addestramento, più che un'architettura diversa. Poiché le politiche rigorosa e permissiva e la soglia derivano dalle stesse tre probabilità, un utente può modificare la nozione di errore senza riaddestrare il modello, cosa che nessun verificatore binario consente.

7.2 Limiti

Una sola piattaforma e una sola famiglia di modelli. Ogni risultato proviene da un unico portatile, un unico runtime e un'unica famiglia di modelli. Latenza ed energia dipendono dall'hardware e dall'implementazione, e ciò che dovrebbe trasferirsi sono i rapporti tra i metodi, non i valori assoluti. Non si effettua alcun confronto di latenza con gli encoder o con il System One generalista, perché questi sono stati eseguiti in runtime diversi e in condizioni che renderebbero il confronto iniquo.

Stabilità dell'addestramento. Le due esecuzioni complete del 0.5B concordano su tutte le metriche aggregate, ma una terza esecuzione è andata precocemente in collasso ed è stata interrotta. La procedura non prevede clipping del gradiente e usa Adam senza correzione del bias, entrambe cause note di fine-tuning instabile [60, 59], per cui ciascuna delle due potrebbe spiegare il collasso; nessuna delle due ipotesi è stata verificata. Il 3B è stato addestrato una sola volta, su metà delle risposte di addestramento, e la sua varianza rispetto al seed è ignota.

Dati di addestramento. RAGTruth è l'unica fonte di addestramento. Le sue contraddizioni sono concentrate nel data-to-text, e al di fuori di questo compito vengono apprese solo in parte. I modelli si affidano inoltre in parte alla sovrapposizione lessicale: un supporto parafrasato appare non supportato, e una piccola modifica in una frase che per il resto copia la fonte passa spesso inosservata. Questo punto cieco si riduce con la dimensione del modello, ma resta il limite principale di entrambi i modelli.

Ambito della decisione. La verifica avviene frase per frase e soltanto rispetto al documento recuperato. Una frase vera nel mondo ma assente dal documento conta come non menzionata, il che è il comportamento corretto per la groundedness ma non per la verifica dei fatti. Le frasi prive di contenuto verificabile sono escluse

da una regola, e la segmentazione stessa può dividere o unire le affermazioni in modo diverso da un lettore umano. Il verdetto su una frase dipende soltanto dal documento e dalle frasi che la precedono.

Ambito statistico. Gli intervalli bootstrap descrivono la variabilità dei dati di test, non quella dell'addestramento, e circa cinquanta confronti sono stati sottoposti a test senza correzione per i test multipli. Le conclusioni poggiano pertanto su effetti ben lontani dallo zero, e non si rivendicano differenze su LLM-AggreFact e VitaminC inferiori alla dispersione tra seed. Diverse baseline sono state valutate su sottocampioni, il che ne allarga gli intervalli.

7.3 Minacce alla validità

Etichette. La distinzione tra contraddetta e non menzionata si fonda sugli annotatori di RAGTruth, il cui accordo sul tipo di allucinazione non è pubblicato, e una recente riannotazione della partizione di test di RAGTruth ha individuato un numero considerevolmente maggiore di risposte allucinate rispetto alle etichette originali [76]. L'insieme di test potrebbe quindi sottostimare le classi minoritarie. Per questo motivo i risultati a tre classi su RAGTruth vanno sempre letti insieme a quelli su VitaminC, le cui etichette hanno un accordo misurato. LLM-AggreFact unifica le etichette di undici dataset di qualità diversa, e alcune delle sue colonne sono notoriamente rumorose.

Vantaggio nel dominio. GroundOne è addestrato su RAGTruth, per cui i suoi risultati sull'insieme di test di RAGTruth e sulla colonna RAGTruth di LLM-AggreFact lo favoriscono rispetto a sistemi addestrati altrove; per converso, VitaminC rientra nella genealogia di addestramento di MiniCheck-RoBERTa. Ogni confronto è riportato con la relativa avvertenza, e le affermazioni poggiano sull'andamento complessivo nei tre insiemi di test piuttosto che su uno solo.

Configurazione delle baseline. I giudici generativi sono sensibili ai prompt. I loro prompt sono stati fissati prima di esaminare i risultati di test, e il prompt del giudice a parità di output è stato scelto sui dati di sviluppo unicamente in base alla validità del formato; l'interpretazione dell'output è stata permissiva a loro favore; la decodifica non era vincolata da una grammatica, il che incide soprattutto sugli errori di formato del modello più piccolo. Non è stato valutato un giudice addestrato sugli stessi dati, né un giudice più grande, per cui il confronto misura ciò che offre un giudice della stessa dimensione senza addestramento. Il System One generalista è stato eseguito con pesi a 4 bit perché il modello a 8 bit non trovava posto accanto al sistema operativo, il che potrebbe sottostimarne l'accuratezza. Il

modello decisionale generalista d1-3B è stato eseguito zero-shot con le domande usate per tutte le baseline System One, che potrebbero non essere la formulazione a esso più congeniale, e su un sottocampione di LLM-AggreFact.

Contaminazione. I modelli di base sono stati preaddestrati su ampi corpora web che potrebbero includere parti degli insiemi di test pubblici. Poiché gli stessi modelli di base sono usati senza addestramento come baseline e raggiungono un'accuratezza vicina al caso sul compito a tre classi, la contaminazione non può spiegare i guadagni dell'addestramento, ma per i valori assoluti non può essere esclusa del tutto.

Capitolo 8

Conclusioni e sviluppi futuri

8.1 Sintesi dei risultati

Questa tesi si è chiesta se un modello linguistico compatto, specializzato in un'unica decisione e impiegato come System One, possa verificare frase per frase le risposte generate con recupero di informazioni su un portatile, e da dove provenga l'eventuale vantaggio. GroundOne, un modello Qwen2.5 addestrato che legge i verdetti di un'intera risposta in un solo passaggio in avanti, fornisce una risposta positiva, ma con alcune riserve.

La specializzazione funziona dove era stata indirizzata. Sull'insieme di test di RAGTruth GroundOne raggiunge un'accuratezza bilanciata a tre classi di 0.769 con 0.5B parametri e di 0.832 con 3B, ben al di sopra degli stessi modelli non addestrati, al di sopra degli encoder specializzati nell'ordinamento delle frasi supportate e nettamente al di sopra di un System One generalista otto volte più grande. Su LLM-AggreFact il 3B è alla pari con il miglior encoder di piccole dimensioni, mentre su VitaminC il vantaggio scompare: uno specialista non è un verificatore di fatti generale. Un modello decisionale generalista della stessa dimensione, addestrato a decidere su un'ampia miscela di compiti, conferma questo compromesso: GroundOne lo supera nettamente su RAGTruth, è alla pari su LLM-AggreFact e resta molto indietro su VitaminC.

Il meccanismo del System One porta vantaggi su due fronti. Leggere tutti i verdetti in un solo passaggio rende la verifica, sulle risposte lunghe, circa due volte più rapida di un giudice generativo sintetico e sei volte più rapida di uno prolisso della stessa dimensione, con un output sempre valido; inoltre, restringere la decisione alle risposte ammissibili è di per sé più accurato che lasciare allo stesso modello la generazione del verdetto. A parità di output, i giudici generativi non sono in grado di fornire probabilità di qualità paragonabile: le loro probabilità separano a malapena le decisioni corrette da quelle errate, e quelle verbalizzate

sono quasi sempre del tipo tutto-o-niente.

Le probabilità di GroundOne possono essere calibrate con una temperatura e un bias stimati sui dati di sviluppo, e una soglia scelta sugli stessi dati le trasforma in etichette coerenti sia con le probabilità sia con la decisione valutata. L'errore di calibrazione atteso, da solo, non distingue tuttavia le probabilità utili da quelle inutili, e la capacità discriminativa va misurata separatamente.

L'intero sistema funziona su un portatile da 16 GB. Una risposta lunga viene verificata in 0.41 secondi e 5.5 joule con il modello 0.5B, e in 2.57 secondi e 33 joule con il 3B. Il risparmio di energia rispetto a un giudice è reale ma inferiore al risparmio di tempo, perché un singolo prefill lungo assorbe più potenza della decodifica.

I limiti principali risiedono nella ristrettezza dei dati di addestramento, a causa della quale le contraddizioni al di fuori del data-to-text e le allucinazioni nascoste in frasi copiate vengono apprese solo in parte, e in una procedura di addestramento non del tutto robusta rispetto al seed casuale.

8.2 Sviluppi futuri

Da questi limiti discendono direttamente diverse direzioni di lavoro. La procedura di addestramento andrebbe resa robusta abilitando la correzione del bias e il clipping del gradiente, registrando le norme dei gradienti e ripetendo le esecuzioni con più seed, anche per il modello 3B. Dati di addestramento con contraddizioni più varie, con supporto parafrasato e con modifiche minime del tipo contenuto in VitaminC dovrebbero affrontare sia il richiamo debole al di fuori del data-to-text sia il punto cieco sulle frasi copiate. Addestrare la lettura dei logit (readout) sulla lettera che il modello scriverebbe spontaneamente dopo il marcatore potrebbe allineare readout e generazione, con effetti sull'accuratezza che restano da misurare.

Sul versante dei sistemi, le misure ancora mancanti per il pilastro della località sono la memoria occupata quando il verificatore opera accanto a un generatore e a un sistema di recupero, e una valutazione completa con la rete disattivata. Un confronto equo della latenza con gli encoder richiede di eseguirli in batch, mentre quello con il System One generalista richiede di confrontare entrambi i sistemi attraverso la stessa interfaccia HTTP. La verifica potrebbe inoltre avvenire durante la generazione della risposta, segnalando ogni frase non appena è completa, cosa che la soluzione a marcatori rende naturale. Infine, un assistente locale costruito sull'interfaccia del servizio consentirebbe agli utenti di scegliere la regola e la soglia più adatte ai propri documenti, ed è proprio in questo contesto che le probabilità calibrate mostrano il loro valore.

Appendice A

Output di ogni sistema su una stessa risposta

Questa appendice mostra che cosa restituisce ciascun sistema per la stessa risposta di T1, così da poter mettere in relazione i numeri del Capitolo 6 con output concreti. La risposta (identificativo `rt-7784`) è una risposta di tipo data-to-text, scritta da un modello linguistico a partire dalla descrizione strutturata di un ristorante: un record JSON che ne riporta il nome, l'indirizzo, le categorie, gli orari di apertura (da martedì a domenica, 11:00–21:00), gli attributi, una valutazione media di 4.5 stelle e due recensioni di clienti. È stata scelta perché tutti i sistemi l'hanno valutata e perché contiene tutte e tre le classi; non è stata selezionata in base alle prestazioni di alcun sistema. Le sue sei frasi, con le relative etichette di riferimento, sono le seguenti:

1. *Mesa Verde is a popular business located in Santa Barbara, CA.* — supportata.
2. *It offers a variety of services including Garage Door Services, Restaurants, Live/Raw Food, . . . , Vegan, and Salad.* — supportata.
3. *The business is open seven days a week from 11 am to 9 pm.* — contraddetta (il lunedì il locale è chiuso).
4. *Mesa Verde has a rating of 4.5 stars on Google and has received multiple positive reviews.* — non menzionata (il record non indica la fonte della valutazione e contiene due recensioni, una delle quali negativa).
5. *One customer praised the restaurant's fresh, delicious, and inventive cuisine, while another was disappointed by the taste and price of the food.* — supportata.
6. *Despite this, many customers have had positive experiences at Mesa Verde and recommend it for its fusion and vegan food options.* — supportata.

A.1 Verdetti frase per frase

La Tabella A.1 riporta il verdetto di ciascun sistema. Per GroundOne l'etichetta è quella restituita, ricavata dalle probabilità calibrate con la soglia stimata sui dati di sviluppo ($\tau = 0.935$ per il modello 0.5B e 0.925 per il 3B), e il numero è la P(supported) calibrata. Per gli encoder il numero è la loro probabilità di supporto, e la decisione è presa a 0.5. S, C e N indicano supportata, contraddetta e non menzionata; un trattino segnala un errore di formato.

Tabella A.1: Verdetti sulla risposta `rt-7784`. Gli errori rispetto al riferimento sono in grassetto.

Sistema	1	2	3	4	5	6
Riferimento	S	S	C	N	S	S
GroundOne 0.5B (restituita, P_{sup})	S 0.96	S 0.98	C 0.50	N 0.05	S 0.98	S 0.97
GroundOne 3B (restituita, P_{sup})	S 0.96	S 0.99	C 0.58	N 0.21	S 0.99	**N 0.85**
B2 0.5B (token generato)	S	S	C	N	S	–
B2 3B (token generato)	S	S	C	–	S	S
0.5B non addestrato (readout)	**C**	**C**	C	**C**	**C**	**C**
Giudice sintetico 0.5B	S	S	**S**	**S**	S	S
Giudice sintetico 3B	**C**	S	**S**	**S**	**N**	S
Giudice verboso 0.5B	–	–	–	–	–	–
Giudice verboso 3B	–	–	–	–	–	–
Giudice a parità di output 3B, logit	**C**	S	**S**	**S**	**N**	S
Giudice a parità di output 3B, probabilità espresse a parole	S	**C**	**N**	**S**	**C**	**N**
MiniCheck-RoBERTa (P_{sup})	0.78	0.90	0.09	0.03	**0.43**	**0.03**
HHEM-2.1-open (P_{sup})	0.75	0.95	0.01	0.09	0.73	**0.24**
Rizzo-Flow 4B	S	S	**S**	**S**	S	S

Il GroundOne da 0.5B classifica correttamente tutte e sei le frasi. La contraddizione della frase 3 viene individuata con una P(supported) calibrata di 0.50: ben al di sotto della soglia, ma lontana dalla certezza, ed è proprio ciò che una probabilità calibrata dovrebbe esprimere per una frase corretta in tutto tranne che per un giorno della settimana. Anche il GroundOne da 3B individua entrambi gli errori, ma restituisce come non menzionata la frase 6, una generalizzazione a partire dalle due recensioni: la sua P(supported) di 0.85 è inferiore alla soglia elevata imposta dal tasso di base di RAGTruth, un caso in cui un utente che preferisca meno segnalazioni abbasserebbe la soglia. A parità di pesi, il token generato (B2) fornisce la lettera corretta su cinque delle sei frasi, ma sulla restante produce uno spazio (0.5B) o la parola "We" (3B) al posto di un verdetto. Gli encoder binari separano bene i due errori, ma respingono anche le frasi 5 o 6, che riassumono le recensioni con parole diverse; Rizzo-Flow accetta ogni frase. I giudici generativi falliscono nei modi quantificati nel Capitolo 6, come mostrano gli output grezzi riportati di seguito.

A.2 Output grezzi

GroundOne 0.5B (JSON restituito, abbreviato). Un unico passaggio in avanti su 1,193 token di input, nessun token generato, 0.47 secondi (2.6 secondi per il 3B):


```
{"design": "b", "threshold": 0.935, "n_decisions": 6,
 "usage": {"input_tokens": 1193, "output_tokens": 0},
 "sentences": [
  {"index": 2, "text": "The business is open seven days a week ...",
   "scored": true, "labels": {"three": "contradicted"},
   "answers": {"three": {"supported": 0.502, "contradicted": 0.462,
                         "not_mentioned": 0.036}},
   "flag_strict": true, "flag_lenient": true},
  {"index": 3, "text": "Mesa Verde has a rating of 4.5 stars ...",
   "scored": true, "labels": {"three": "not_mentioned"},
   "answers": {"three": {"supported": 0.051, "contradicted": 0.003,
                         "not_mentioned": 0.946}},
   "flag_strict": true, "flag_lenient": false}, ...]}
```


Giudice sintetico 3B. Una riga per frase, come richiesto, ma con lettere errate per quattro frasi:

```
1: B
2: A
3: A
4: A
5: C
6: A
```

Giudice verboso 3B (primi tre verdetti). Il ragionamento riformula la frase e cita il documento come supporto anche per la frase contraddetta e per quella non supportata, e ogni verdetto è "A". I verdetti sono scritti su una riga separata anziché sulla stessa riga, come chiedeva l'istruzione, per cui il protocollo li conta come errori di formato:

```
1: Mesa Verde is located in Santa Barbara, CA, as stated in the document.
Verdict: A

2: Mesa Verde offers a variety of services including Garage Door Services,
   [...], as mentioned in the categories provided.
Verdict: A

3: The business is open seven days a week from 11 am to 9 pm, as indicated
   by the hours provided.
Verdict: A
```

Giudice verboso 0.5B. Ricopia le sei frasi, numerate, senza alcun verdetto.

Giudice a parità di output 3B, probabilità espresse a parole. Le probabilità sono tutte one-hot, e per le frasi 3 e 6 l'etichetta dichiarata (C) contraddice le probabilità dichiarate, che concentrano tutta la massa su A:


```
{"1": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "2": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "3": {"label": "C", "p": [1.0, 0.0, 0.0]},
 "4": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "5": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "6": {"label": "C", "p": [1.0, 0.0, 0.0]}}
```


Giudice a parità di output 0.5B, probabilità espresse a parole. Lo stesso vettore per ogni frase:


```
{"1": {"label": "A", "p": [0.3, 0.3, 0.3]},
 "2": {"label": "A", "p": [0.3, 0.3, 0.3]},
 ...
 "6": {"label": "A", "p": [0.3, 0.3, 0.3]}}
```

Bibliografia

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020 (cit. alle pp. 1, 9).

[2] Georgi Gerganov e ggml-org contributors. *llama.cpp: LLM inference in C/C++*. 2023. URL: `https://github.com/ggml-org/llama.cpp` (cit. alle pp. 1, 6, 8, 30).

[3] Apple. *MLX: An array framework for Apple silicon*. 2023. URL: `https://github.com/ml-explore/mlx` (cit. alle pp. 1, 7, 16).

[4] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu et al. "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. alle pp. 1, 9, 19, 20).

[5] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong et al. "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions". In: *ACM Transactions on Information Systems* (2025). DOI: `10.1145/3703155` (cit. alle pp. 1, 9).

[6] TypeSafe. *Introducing System One Models and Jev*. Blog post. 2026. URL: `https://typesafe.ai/blog/introducing-system-one-models-and-jev` (cit. alle pp. 1, 7).

[7] TheoLeeCJ. *SemIf: open option-logit System One baseline*. 2026. URL: `https://github.com/TheoLeeCJ/SemIf` (cit. alle pp. 2, 7).

[8] Simone Rizzo. *Rizzo Flow: typed decisions from an LLM, without generating a single token*. 2026. URL: `https://github.com/Rizzo-AI-Academy/rizzo-flow` (cit. alle pp. 2, 7, 30).

[9] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin et al. "Efficiently Scaling Transformer Inference". In: *Proceedings of Machine Learning and Systems (MLSys)*. 2023 (cit. alle pp. 5, 13).

[10] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah et al. "Splitwise: Efficient Generative LLM Inference Using Phase Splitting". In: *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024 (cit. alle pp. 5–7, 13, 46).

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. 2023 (cit. a p. 6).

[12] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard et al. "LLM in a flash: Efficient Large Language Model Inference with Limited Memory". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. a p. 6).

[13] Apple. *MacBook Pro (14-inch, M4, 2024) – Technical Specifications*. URL: `https://support.apple.com/en-us/121552` (visitato il giorno 2026-10-10) (cit. alle pp. 6, 28).

[14] Apple. *mlx-lm: LLMs with MLX*. 2024. URL: `https://github.com/ml-explore/mlx-lm` (cit. alle pp. 7, 16).

[15] Tom Gunter, Zirui Wang, Chong Wang, Ruoming Pang et al. *Apple Intelligence Foundation Language Models*. 2024. arXiv: `2407.21075` (cit. a p. 7).

[16] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li et al. "From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference". In: *IEEE High Performance Extreme Computing Conference (HPEC)*. 2023 (cit. alle pp. 7, 46).

[17] Alexandra Sasha Luccioni, Yacine Jernite e Emma Strubell. "Power Hungry Processing: Watts Driving the Cost of AI Deployment?" In: *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. 2024 (cit. alle pp. 7, 46).

[18] Jacky Kwok, Hangoo Kang, Tarun Suresh, Jon Saad-Falcon, Marco Pavone, Christopher Ré e Azalia Mirhoseini. *Contrastive Language Models: A System One Model for Fast and Generalizable Decision-Making*. Blog post, not peer-reviewed. 2026. URL: `https://contrastive-lm.notion.site` (cit. a p. 8).

[19] Convai Innovations. *Laya: a System One decision model (model card, convaiinnovations/laya)*. URL: `https://huggingface.co/convaiinnovations/laya` (visitato il giorno 2026-10-10) (cit. a p. 8).

[20] Seyedarmin Azizi, Erfan Baghaei Potraghloo e Massoud Pedram. *Labels Override Definitions in Jev-Style Typed Decision Models*. 2026. arXiv: `2610.02586` (cit. a p. 8).

[21] Tobias Deußer, Lorenz Sparrenberg e Rafet Sifa. *Evaluating and Benchmarking the System One Model Jev*. 2026. arXiv: 2609.37647 (cit. alle pp. 8, 40).

[22] Gowthamkumar Nandakishore. *Laya as a Typed Probabilistic Assessor: An Independent Reproduction and a Preregistered Study of Calibration and Selective Escalation*. 2026. arXiv: 2609.33843 (cit. a p. 8).

[23] Cho Yin Yong. *DecideBench (dataset card, choyiny/decidebench)*. URL: https://huggingface.co/datasets/choyiny/decidebench (visitato il giorno 2026-10-10) (cit. a p. 8).

[24] Hugging Face. *Models tagged decision-model*. URL: https://huggingface.co/models?other=decision-model (visitato il giorno 2026-10-10) (cit. a p. 8).

[25] Cloudflare. *Clef: open decision models (model card, Cloudflare/clef)*. URL: https://huggingface.co/Cloudflare/clef (visitato il giorno 2026-10-10) (cit. a p. 8).

[26] Liquid AI. *d1-3B: a decision model built on LFM2.5-VL-3B (model card, LiquidAI/d1-3B, revision 051bcc4)*. URL: https://huggingface.co/LiquidAI/d1-3B (visitato il giorno 2026-10-10) (cit. alle pp. 8, 30).

[27] Perplexity. *pplx-decider-v1.1-27b (model card)*. URL: https://huggingface.co/perplexity-ai/pplx-decider-v1.1-27b (visitato il giorno 2026-10-10) (cit. a p. 8).

[28] multimodalart. *Jev Decision Index: a leaderboard of System One decision models (Hugging Face Space, v0.2.1)*. URL: https://huggingface.co/spaces/multimodalart/jev-decision-index (visitato il giorno 2026-10-10) (cit. alle pp. 8, 37).

[29] Ruoqi Guo, Yi Liu, Gelei Deng, Yuekang Li et al. *Just Ask Jev: Reinforcement Learning for Calibrated Decisions as a Zero-Shot Detector of AI Alignment Failures*. 2026. arXiv: 2609.29429 (cit. a p. 8).

[30] Yubo Li, Yidi Miao, Ramayya Krishnan e Rema Padman. *JEV-as-a-Judge: Accept When Confident, Escalate When Unsure*. 2026. arXiv: 2609.26550 (cit. a p. 8).

[31] Xinpeng Wang, Bolei Ma, Chengzhi Hu, Leon Weber-Genzel et al. ""My Answer is C": First-Token Probabilities Do Not Match Text Answers in Instruction-Tuned Language Models". In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024. arXiv: 2402.14499 (cit. alle pp. 9, 14).

[32] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou e Minlie Huang. "Large Language Models Are Not Robust Multiple Choice Selectors". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. alle pp. 9, 14).

[33] Adina Williams, Nikita Nangia e Samuel R. Bowman. "A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference". In: *Proceedings of NAACL-HLT 2018*. 2018 (cit. a p. 9).

[34] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston e Douwe Kiela. "Adversarial NLI: A New Benchmark for Natural Language Understanding". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020 (cit. alle pp. 9, 20).

[35] Tal Schuster, Adam Fisch e Regina Barzilay. "Get Your Vitamin C! Robust Fact Verification with Contrastive Evidence". In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 624–643. DOI: 10.18653/v1/2021.naacl-main.52. URL: https://aclanthology.org/2021.naacl-main.52 (cit. alle pp. 9, 19, 20).

[36] Philippe Laban, Tobias Schnabel, Paul N. Bennett e Marti A. Hearst. "SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization". In: *Transactions of the Association for Computational Linguistics* 10 (2022), pp. 163–177 (cit. a p. 10).

[37] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis et al. "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation". In: *Proceedings of EMNLP 2023*. 2023 (cit. a p. 10).

[38] Yuheng Zha, Yichi Yang, Ruichen Li e Zhiting Hu. "AlignScore: Evaluating Factual Consistency with a Unified Alignment Function". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. alle pp. 10, 40).

[39] Liyan Tang, Philippe Laban e Greg Durrett. "MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents". In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2024 (cit. alle pp. 10, 19, 20, 31).

[40] Vectara. *HHEM-2.1-Open: Hughes Hallucination Evaluation Model*. 2024. URL: https://huggingface.co/vectara/hallucination_evaluation_model (cit. alle pp. 10, 31).

[41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2023 (cit. a p. 10).

[42] Shahul Es, Jithin James, Luis Espinosa-Anke e Steven Schockaert. "RAGAs: Automated Evaluation of Retrieval Augmented Generation". In: *Proceedings of EACL 2024: System Demonstrations*. 2024 (cit. a p. 10).

[43] Jon Saad-Falcon, Omar Khattab, Christopher Potts e Matei Zaharia. "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems". In: *Proceedings of NAACL 2024*. 2024 (cit. a p. 10).

[44] Selvan Sunitha Ravi, Bartosz Mielczarek, Anand Kannappan, Douwe Kiela e Rebecca Qian. *Lynx: An Open Source Hallucination Evaluation Model*. 2024. arXiv: 2407.08488 (cit. a p. 10).

[45] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil e Hannaneh Hajishirzi. "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. a p. 10).

[46] Brandon T. Willard e Rémi Louf. *Efficient Guided Generation for Large Language Models*. 2023. arXiv: 2307.09702 (cit. alle pp. 10, 30).

[47] Liyan Tang, Philippe Laban e Greg Durrett. *LLM-AggreFact Leaderboard*. 2024. URL: https://llm-aggrefact.github.io/ (visitato il giorno 2026-10-05) (cit. alle pp. 10, 19, 20).

[48] Chuan Guo, Geoff Pleiss, Yu Sun e Kilian Q. Weinberger. "On Calibration of Modern Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2017 (cit. alle pp. 10, 15).

[49] Mahdi Pakdaman Naeini, Gregory F. Cooper e Milos Hauskrecht. "Obtaining Well Calibrated Probabilities Using Bayesian Binning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2015 (cit. alle pp. 10, 15).

[50] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein e Sameer Singh. "Calibrate Before Use: Improving Few-Shot Performance of Language Models". In: *International Conference on Machine Learning (ICML)*. 2021 (cit. alle pp. 10, 15).

[51] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan et al. *Language Models (Mostly) Know What They Know*. 2022. arXiv: 2207.05221 (cit. a p. 10).

[52] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma et al. "Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback". In: *Proceedings of EMNLP 2023*. 2023 (cit. alle pp. 11, 43).

[53] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li et al. "Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. alle pp. 11, 43).

[54] Yonatan Geifman e Ran El-Yaniv. "Selective Classification for Deep Neural Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017 (cit. alle pp. 11, 32).

[55] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang e Weizhu Chen. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2022 (cit. alle pp. 11, 14).

[56] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman e Luke Zettlemoyer. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023 (cit. alle pp. 11, 14).

[57] Qwen Team. *Qwen2.5 Technical Report*. 2024. arXiv: `2412.15115` (cit. alle pp. 11, 14, 28).

[58] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi e Noah Smith. *Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping*. 2020. arXiv: `2002.06305` (cit. alle pp. 11, 29).

[59] Marius Mosbach, Maksym Andriushchenko e Dietrich Klakow. "On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines". In: *International Conference on Learning Representations (ICLR)*. 2021 (cit. alle pp. 11, 36, 50).

[60] Razvan Pascanu, Tomas Mikolov e Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2013 (cit. alle pp. 11, 36, 50).

[61] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder e Li Deng. *MS MARCO: A Human Generated MAchine Reading COmprehension Dataset*. CoCo Workshop at NIPS 2016. 2016. arXiv: `1611.09268` (cit. a p. 19).

[62] Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman e Phil Blunsom. "Teaching Machines to Read and Comprehend". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015 (cit. a p. 19).

[63] Abigail See, Peter J. Liu e Christopher D. Manning. "Get To The Point: Summarization with Pointer-Generator Networks". In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2017 (cit. a p. 19).

[64] Yelp. *Yelp Open Dataset*. URL: `https://business.yelp.com/data/resources/open-dataset/` (visitato il giorno 2026-10-05) (cit. a p. 19).

[65] Liyan Tang, Tanya Goyal, Alex Fabbri, Philippe Laban, Jiacheng Xu, Semih Yavuz, Wojciech Kryściński, Justin Rousseau e Greg Durrett. "Understanding Factual Errors in Summarization: Errors, Summarizers, Datasets, Error Detectors". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. a p. 20).

[66] Liyan Tang, Igor Shalyminov, Amy Wing-mei Wong et al. "TofuEval: Evaluating Hallucinations of LLMs on Topic-Focused Dialogue Summarization". In: *Proceedings of NAACL 2024*. 2024 (cit. a p. 20).

[67] Nelson F. Liu, Tianyi Zhang e Percy Liang. "Evaluating Verifiability in Generative Search Engines". In: *Findings of EMNLP 2023*. 2023 (cit. a p. 20).

[68] Chaitanya Malaviya, Subin Lee, Sihao Chen, Elizabeth Sieber, Mark Yatskar e Dan Roth. "ExpertQA: Expert-Curated Questions and Attributed Answers". In: *Proceedings of NAACL 2024*. 2024 (cit. a p. 20).

[69] Hung-Ting Chen, Fangyuan Xu, Shane Arora e Eunsol Choi. "Understanding Retrieval Augmentation for Long-Form Question Answering". In: *Conference on Language Modeling (COLM)*. 2024 (cit. a p. 20).

[70] Ryo Kamoi, Tanya Goyal, Juan Diego Rodriguez e Greg Durrett. "WiCE: Real-World Entailment for Claims in Wikipedia". In: *Proceedings of EMNLP 2023*. 2023 (cit. a p. 20).

[71] Alon Jacovi, Yonatan Bitton, Bernd Bohnet, Jonathan Herzig, Or Honovich, Michael Tseng, Michael Collins, Roee Aharoni e Mor Geva. "A Chain-of-Thought Is as Strong as Its Weakest Link: A Benchmark for Verifiers of Reasoning Chains". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. a p. 20).

[72] Yuxia Wang, Revanth Gangi Reddy, Zain Muhammad Mujahid et al. "Factcheck-Bench: Fine-Grained Evaluation Benchmark for Automatic Fact-checkers". In: *Findings of EMNLP 2024*. 2024 (cit. a p. 20).

[73] Liyan Tang. *LLM-AggreFact dataset card (lytang/LLM-AggreFact)*. URL: `https://huggingface.co/datasets/lytang/LLM-AggreFact` (visitato il giorno 2026-10-05) (cit. a p. 20).

[74] Tal Schuster. *VitaminC dataset card (tals/vitaminc)*. URL: `https://huggingface.co/datasets/tals/vitaminc` (visitato il giorno 2026-10-06) (cit. a p. 20).

[75] Nipun Sadvilkar e Mark Neumann. "PySBD: Pragmatic Sentence Boundary Disambiguation". In: *Proceedings of the Second Workshop for NLP Open Source Software (NLP-OSS)*. 2020, pp. 110–114. DOI: `10.18653/v1/2020.nlposs-1.15`. URL: `https://aclanthology.org/2020.nlposs-1.15` (cit. a p. 21).

[76] Boxi Yu, Yuzhong Zhang, Liting Lin, Lionel Briand e Emir Muñoz. *Retromorphic Testing with Hierarchical Verification for Hallucination Detection in RAG*. 2026. arXiv: 2603.27752. URL: https://arxiv.org/abs/2603.27752 (cit. alle pp. 24, 51).

[77] Philipp Koehn. "Statistical Significance Tests for Machine Translation Evaluation". In: *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2004, pp. 388–395. URL: https://aclanthology.org/W04-3250 (cit. a p. 32).

[78] Taylor Berg-Kirkpatrick, David Burkett e Dan Klein. "An Empirical Investigation of Statistical Significance in NLP". In: *Proceedings of EMNLP-CoNLL 2012*. 2012, pp. 995–1005. URL: https://aclanthology.org/D12-1091 (cit. a p. 32).

[79] Rotem Dror, Gili Baumer, Segev Shlomov e Roi Reichart. "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing". In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2018, pp. 1383–1392. URL: https://aclanthology.org/P18-1128 (cit. a p. 32).